

Contents

	Page
1 Introduction	1
1.1 What Is L ^A T _E X?	1
1.2 Why L ^A T _E X Over Other Word Processors?	1
1.3 How to Prepare a L ^A T _E X Input File?	2
1.4 How to Compile a L ^A T _E X Input File?	4
1.5 L ^A T _E X Syntax	5
1.5.1 Commands	5
1.5.2 Environments	6
1.5.3 Packages	6
1.6 Keyboard Characters in L ^A T _E X	7
1.7 How to Read this Book?	8
2 Fonts Selection	9
2.1 Text-Mode Fonts	9
2.2 Math-Mode Fonts	11
2.3 Emphasized Fonts	12
2.4 Colored Fonts	13
3 Formatting Texts I	15
3.1 Sectional Units	15
3.2 Labeling and Referring Numbered Items	16
3.3 Texts Alignment	18
3.4 Quoted Texts	18
3.5 New Lines and Paragraphs	19
3.5.1 Creating New Lines	19
3.5.2 Creating New Paragraphs	20
3.6 Creating and Filling Blank Space	21
3.7 Producing Dashes Within Texts	24
3.8 Preventing Line Break*	24
3.9 Adjusting Blank Space After a Period Mark*	24
3.10 Hyphenating a Word*	25

4	Formatting Texts II.	27
4.1	Increasing Depth of Sectional Units*	27
4.2	Changing Titles and Counters of Sectional Units*	28
4.3	Multiple Columns.	29
4.3.1	Multiple Columns Related Parameters	30
4.3.2	A Flexible Approach to Generate Multiple Columns	30
4.4	Mini Pages	31
4.5	Foot Notes	32
4.5.1	Foot Notes in Mini Pages*	33
4.5.2	Altering the Pattern of Foot Notes*	34
4.6	Marginal Notes*	35
5	Page Layout and Style.	37
5.1	Page Layout	37
5.1.1	Standard Page Layout	37
5.1.2	Formatting Page Layout*	38
5.1.3	Increasing the Height of a Page*	39
5.2	Page Style.	40
5.3	Running Header and Footer.	40
5.3.1	Header with the headings Style	41
5.3.2	Header with the myheadings Style.	42
5.3.3	Header and Footer with the fancy Style Under the fancyheadings Package*	43
5.3.4	Header and Footer with the fancy Style Under the fancyhdr Package*	45
5.4	Page Breaking and Adjustment	46
5.5	Page Numbering	46
6	Listing and Tabbing Texts.	49
6.1	Listing Texts	49
6.1.1	Numbered Listing Through the enumerate Environment	49
6.1.2	Unnumbered Listing Through the itemize Environment	53
6.1.3	Listing with User-Defined Labels Through the description Environment	54
6.1.4	Nesting Different Listing Environments	55
6.1.5	Indentation of Listed Items*	56
6.2	Tabbing Texts Through the tabbing Environment.	57
6.2.1	Adjusting Column Width in the tabbing Environment	57
6.2.2	Adjusting Alignment of Columns in the tabbing Environment*	58
7	Table Preparation I.	59
7.1	Table Through the tabular Environment	59
7.2	Table Through the tabularx Environment.	60
7.3	Vertical Positioning of Tables	62
7.4	Sideways (Rotated) Texts in Tables*	62
7.5	Adjusting Column Width in Tables*	63
7.6	Additional Provisions for Customizing Columns of Tables*	64
7.7	Merging Rows and Columns of Tables	66
7.8	Table Wrapped by Texts*	68
7.9	Table with Colored Background*	68

8	Table Preparation II	71
8.1	Nested Tables*	71
8.2	Column Alignment About Decimal Point*	71
8.3	Side-by-Side Tables*	73
8.4	Sideways (Rotated) Table*	75
8.5	Long Table on Multiple Pages*	76
8.6	Tables in Multi-column Documents	78
8.7	Foot Notes in Tables*	78
8.8	Changing Printing Format of Tables*	79
8.9	Tables at the End of a Document	80
9	Figure Insertion	81
9.1	Commands and Environment for Inserting Figures	81
9.2	Inserting a Simple Figure	82
9.3	Side-by-Side Figures*	83
9.4	Sub-numbering a Group of Figures	85
9.5	Figure Wrapped by Texts*	86
9.6	Rotated Figure	87
9.7	Mathematical Notations in Figures*	87
9.8	Figures in Tables*	89
9.9	Figures in Multi-column Documents	89
9.10	Changing Printing Format of Figures*	89
9.11	Figures at the End of a Document	90
9.12	Editing L ^A T _E X Input File Involving Many Figures*	90
10	Figure Drawing	91
10.1	Circles and Circular Arcs	92
10.2	Straight Lines and Vectors*	93
10.3	Curves*	95
10.4	Oval Boxes*	96
10.5	Texts in Figures*	97
10.6	Compound Figures*	100
11	Equation Writing I	101
11.1	Basic Mathematical Notations and Delimiters	101
11.2	Mathematical Operators	103
11.3	Mathematical Expressions in Text-Mode	104
11.4	Simple Equations	104
11.4.1	Eliminating Equation Numbering	105
11.4.2	Overwriting Equation Numbering	105
11.4.3	Changing Printing Format of Equations*	105
11.5	Array of Equations	107
11.6	Left Aligning an Equation*	109
11.7	Sub-numbering a Set of Equations*	111
12	Equation Writing II	113
12.1	Texts and Blank Space in Math-Mode	113
12.2	Conditional Expression	114
12.3	Evaluation of Functional Values	115
12.4	Splitting an Equation into Multiple Lines*	115

12.5	Vector and Matrix	117
12.6	Overlining and Underlining	119
12.7	Stacking Terms*	120
12.8	Side-by-Side Equations*	123
13	User-Defined Macros	125
13.1	Defining New Commands	125
13.1.1	New Commands Without Argument	126
13.1.2	New Commands with Mandatory Arguments	126
13.1.3	New Commands with Optional Arguments	128
13.2	Redefining Existing Commands*	128
13.3	Defining New Environments	130
13.3.1	New Environments Without Argument	130
13.3.2	New Environments with Arguments	131
13.3.3	Theorem-Like Environments	131
13.3.4	Floating Environments for Textual Materials*	133
13.4	Redefining Existing Environments*	135
14	Bibliography with L^AT_EX	137
14.1	Preparation of Bibliographic Reference Database	137
14.2	Citing Bibliographic References	138
14.3	Compiling thebibliography Based L ^A T _E X Input File	140
15	Bibliography with B_IB_TE_X Program	141
15.1	Preparation of B _I B _T E _X Compatible Reference Database	141
15.2	Standard Bibliographic Styles of L ^A T _E X	146
15.3	Use of the natbib Package	147
15.4	Compiling B _I B _T E _X based L ^A T _E X Input File	149
15.5	Editing the .bbl File*	149
15.6	Multiple Bibliographies*	150
16	Lists of Contents and Index	153
16.1	Lists of Contents	153
16.1.1	Information to the Lists of Contents	153
16.1.2	Formatting Lists of Contents*	154
16.1.3	Multiple Lists of Contents*	156
16.1.4	Compiling L ^A T _E X Input File Having Lists of Contents	158
16.2	Making Index	158
16.2.1	Indexing Terms	159
16.2.2	Some Guidelines on Indexing	160
16.2.3	Compiling a L ^A T _E X Input File Having Index	160
17	Miscellaneous I	161
17.1	Boxed Items	161
17.1.1	Texts in Plain Boxes	161
17.1.2	Texts in Color Boxes	162
17.1.3	Mathematical Expressions in Boxes	163
17.1.4	Paragraphs in Boxes*	163
17.1.5	Set of Items in a Box	164
17.2	Rotated Items*	166

17.3	Items at Different Levels and Forms*	167
17.4	Geometric Transformation of Items*	169
18	Miscellaneous II	171
18.1	Horizontal Rules and Dots.	171
18.2	Hyperlinking Referred and Cited Items	171
18.3	Current Date and Time*	172
18.4	Highlighted Texts*	173
18.5	Verbatim Texts	173
18.5.1	Boxed and Listed Verbatim Texts.	174
18.5.2	Verbatim Texts with $\LaTeX$ Commands*	174
18.6	Fragile Commands	176
18.7	Watermarking on Pages*	177
18.8	Logo in Header and Footer*	178
18.9	Paragraphs in Different Forms*	178
19	Letter and Article	181
19.1	Letter Writing	181
19.2	Article Preparation	182
19.2.1	List of Authors.	185
19.2.2	Title and Abstract on Separate Pages.	186
19.2.3	Left Aligned Title and List of Authors*	186
19.2.4	Articles in Multiple Columns	187
19.2.5	Section-Wise Numbering of Items*	189
19.2.6	Dividing an Article into Parts*	189
20	Book and Report	191
20.1	Template of a Book	191
20.2	Book Preparation Using a Root File	192
20.3	Dividing a Book into Parts*	198
20.4	Compilation of a Book	198
20.4.1	Executable File for Compiling a Book*	199
20.4.2	Partial Compilation of a Book*	201
21	Slide Preparation I	203
21.1	Frames in Presentation	203
21.2	Sectional Units in Presentation.	205
21.3	Presentation Structure	205
21.3.1	Title Page	207
21.3.2	Presentation Contents	208
21.3.3	Bibliographic Reference Page.	209
21.4	Appearance of a Presentation (BEAMER Themes)	209
21.4.1	Presentation Theme.	209
21.4.2	Color Theme*	210
21.4.3	Font Theme*	212
21.4.4	Inner Theme*	212
21.4.5	Outer Theme*	213
21.5	Frame Customization*	213
21.5.1	Logo in Frames	214
21.5.2	Font Type	214

21.5.3	Frame Size.	214
21.5.4	Frame Shrinking.	215
21.5.5	Removal of Headline/Footline and Sidebar.	215
21.5.6	Frame Breaking	216
22	Slide Preparation II.	217
22.1	Piece-Wise Presentation (BEAMER Overlays)	217
22.1.1	Table of Contents	217
22.1.2	Uncovering Sequentially Using the <code>\pause</code> Command	218
22.1.3	Uncovering Sequentially Using the Incremental Specification <code><+></code>	218
22.1.4	Other Piece-Wise Presentation Specifications*	219
22.2	Environments in BEAMER Class*	223
22.3	Table and Figure in Presentation*	224
22.4	Dividing a Frame Column-Wise*	226
22.5	Repeating Slides in Presentation*	227
22.6	Jumping (Hyperlink) to Other Slides*	227
23	Error and Warning Messages	231
23.1	Error Message	232
23.2	Warning Message.	237
23.3	Error Without Any Message	238
23.4	Tips for Debugging	239
24	Exercise	241
Appendix A: Symbols and Notations		247
A.1	Text-Mode Accents and Symbols	247
A.2	Math-Mode Symbols	248
Bibliography.		255
Index		257

List of Tables

	Page
1 Introduction	1
1.1 Standard options to the <code>\documentclass[]</code> command	3
1.2 A simple LaTeX input file and its output	4
1.3 Keyboard characters that can be produced directly	7
1.4 Keyboard characters to be produced through commands	8
2 Fonts Selection	9
2.1 Different types of text-mode fonts used in LaTeX	10
2.2 Different types of math-mode fonts used in LaTeX	11
2.3 Various forms of emphasized texts under the <code>ulem</code> package	13
2.4 Predefined as well as some user-defined colors for fonts	14
3 Formatting Texts I	15
3.1 Labeling and referring numbered items	17
3.2 User-defined alignments of texts.	18
3.3 Quoted texts in a narrowed width and specified line spacing	19
3.4 Creating new lines	20
3.5 Creating new paragraphs	21
3.6 Creating blank spaces	22
3.7 Applications of some blank space creating commands.	22
3.8 Dashes of different lengths	24
3.9 Maintaining proper gap after a period (full-stop) mark	25
4 Formatting Texts II.	27
4.1 Changing titles of sectional units	28
4.2 Two columns separated by a vertical line through the <code>\columnseprule</code> command.	30
4.3 Multiple columns generated through the <code>multicols</code> environment	31
4.4 Dividing a page width-wise using the <code>minipage</code> and <code>boxedminipage</code> environments.	32
4.5 Foot notes generated through the <code>\footnote{}</code> command.	33

4.6	Foot notes in mini pages	34
4.7	Foot notes of a mini page to work like those in main pages	35
5	Page Layout and Style.	37
5.1	Types and sizes of standard papers accepted by the \documentclass[]{} command	37
5.2	Commands controlling some important parameters of a page layout	38
5.3	A manually defined page layout	40
5.4	Page styles to control running header and footer in a document	41
5.5	Styles of headers under the headings page style.	41
5.6	Redefining running header generating marker commands.	42
5.7	Commands for headers and footers under the fancyheadings package.	43
5.8	Header and footer with the fancy page style under the fancyheadings package	44
5.9	Header and footer with the fancy page style under the fancyhdr package.	45
5.10	Different types of page numbering	47
6	Listing and Tabbing Texts.	49
6.1	Numbered listing through the enumerate environment	49
6.2	Nested numbered listing through the enumerate environment	50
6.3	Altering styles of numbered listing under the enumerate environment	51
6.4	Numbered listing under the enumerate environment mixed with global fixed texts	52
6.5	Numbered listing under the enumerate environment mixed with local fixed texts	53
6.6	Unnumbered listing through the itemize environment	53
6.7	Nested unnumbered listing through the itemize environment	54
6.8	Altering styles of unnumbered listing under the itemize environment	54
6.9	Listing with user-defined labels through the description environment	55
6.10	Nested different listing environments	56
6.11	Tabbing texts in different columns through the tabbing environment	57
6.12	Adjusting tabbing column width in the tabbing environment through the \hspace{} command	57
6.13	Adjusting tabbing column width in the tabbing environment using the \kill command	58
6.14	Aligning tabbing texts in the tabbing environment using \ and \	58
7	Table Preparation I.	59
7.1	A simple table through the tabular environment	59
7.2	A simple table through the tabularx environment	61
7.3	Table with entries in vertical direction	63
7.4	Fixing column widths in tables with p{} , m{} , and b{}	64

7.5	Some additional provisions for customizing a table.	66
7.6	Merging two or more cells of a table into a single one	67
7.7	Table wrapped by texts through the wraptable environment.	68
7.8	Table with colored background through the \rowcolor{} , \columncolor{} , and \cellcolor{} commands	69
8	Table Preparation II	71
8.1	Nesting two or more tables	72
8.2	Aligning columns of a table about decimal marks.	72
8.3	Side-by-side tables through consecutive tabular environments	73
8.4	Side-by-side tables through the minipage environment	74
8.5	Rotated table through the sideways environment	75
8.6	Rotated table on landscape page through the sidewaysstable environment	76
8.7	Long table in multiple pages through the longtable environment	77
8.8	Foot notes in a table under the tabular environment	79
9	Figure Insertion	81
9.1	L ^A T _E X compilation commands and supported figure formats	81
9.2	Figure insertion through the \epsfig{} command	83
9.3	Side-by-side figures in a single row	84
9.4	Side-by-side figures through the minipage environment.	84
9.5	Sub-numbering a group of figures using the \subfigure[]{} command	85
9.6	Figure wrapped by texts	86
9.7	Rotated figure on a landscape page through the sidewaysfigure environment	87
9.8	Mathematical notations in figures through the \psfrag{} command	87
9.9	Mathematical notations in figures through xfig software	88
10	Figure Drawing	91
10.1	Circle and circular arc drawing	93
10.2	Straight line and vector (arrow) drawing	94
10.3	Open and closed curves, and Bézier quadratic curves	95
10.4	Options to the \oval() command for drawing partial ovals.	96
10.5	Oval boxes through the \oval() command	96
10.6	Settings of the \makebox() , \framebox() and \dashbox() commands for positioning texts in figures	97
10.7	Texts in figures through the \makebox() command	98
10.8	Boxed texts in figures using the \framebox() and \dashbox() commands	99
10.9	Multi-line texts in a figure through the \parbox() command	99
10.10	Rotated texts in figures through the \rotatebox() command.	100
10.11	Compound figures through multiple commands	100

11 Equation Writing I	101
11.1 Frequently used mathematical notations (math-mode)	102
11.2 Basic delimiters (math-mode)	102
11.3 Basic binary operators	103
11.4 Basic relation operators	103
11.5 A simple equation through the equation environment	104
11.6 Different approaches for producing equations without numbering	105
11.7 Overwriting equation numbering by the \tag*{} command	105
11.8 Changing the standard format for printing and numbering equations	106
11.9 Array of equations producing environments and their alignment structures	107
11.10 Array of equations in different forms	108
11.11 Left aligned equations through the flalign and flalign* environments	110
11.12 Sub-numbering a set of equations	111
12 Equation Writing II	113
12.1 Normal texts and gap in math-mode	114
12.2 Conditional mathematical expressions in different forms	114
12.3 Evaluation of a function for a given value	115
12.4 Splitting an equation into multiple lines through the multline environment	116
12.5 Splitting an equation into multiple lines through the split environment	116
12.6 Mathematical analysis through the split environment	116
12.7 Splitting an equation into multiple lines through the eqnarray environment	117
12.8 Matrices and vectors through direct environments	118
12.9 Matrices and vectors through the array environment	118
12.10 Matrix and vector mixed expression through the array environment	119
12.11 Mathematical expression overlined through the \overline{} command	119
12.12 Mathematical expression with over and under braces	120
12.13 Stacking a mathematical term with an arrow of fixed length	120
12.14 Stacking a mathematical term with an over or under arrow of flexible length	121
12.15 Stacking two mathematical terms above and below an arrow	122
12.16 Stacking multiple mathematical lines above or below of a symbol	122
12.17 Side-by-side equations along the page width	123
13 User-Defined Macros	125
13.1 Defining new commands without argument	126
13.2 Definition and application of new commands with mandatory arguments	127
13.3 Definition and application of new commands with optional arguments	128
13.4 Heading commands and their default label-words	129
13.5 Definition and application of new environments without argument	130
13.6 Defining theorem-like environments	132

13.7	Application of the user-defined ‘Definition’ environment	132
13.8	Floats for presenting algorithms and computer programs	134
13.9	Redefining the unnumbered listing environment itemize	135
14	Bibliography with L^AT_EX	137
14.1	Bibliographic references through the thebibliography environment . . .	138
14.2	Bibliographic reference database compatible to the thebibliography environment	139
14.3	Citing bibliographic references of the thebibliography environment.	139
15	Bibliography with the B_IB_TE_X Program.	141
15.1	Types and fields of references under the B _I B _T E _X program	142
15.2	B _I B _T E _X program compatible bibliographic reference database	145
15.3	Some standard bibliographic styles of L ^A T _E X	146
15.4	Some bibliographic styles defined in the natbib package	148
15.5	Citation commands provided under the natbib package	148
15.6	Citation patterns under the natbib package.	149
16	Lists of Contents and Index	153
16.1	Book with chapter-wise lists of Contents	156
16.2	Book with chapter-wise lists of Contents (output of the input file of Table 16.1)	157
16.3	Document having index	159
17	Miscellaneous I	161
17.1	Single-line texts in boxes	162
17.2	Equations in boxes.	163
17.3	Paragraphs in boxes through the \parbox[]{}{} command.	164
17.4	Array of equations in a box through the tabular environment in \shabox{}	165
17.5	Array of equations in a box through the Beqnarray environment in \shabox{}	165
17.6	Unnumbered list in a box through the Bitemize environment in \shabox{}	166
17.7	Rotated items through the rotate environment	166
17.8	Rotated items through the turn environment.	167
17.9	Texts at different levels and forms through the \raisebox{}[]{}{} command	168
17.10	Geometric transformation of texts.	169
18	Miscellaneous II	171
18.1	Manually formatted texts through the verbatim environment	173
18.2	Line numbering of verbatim texts through the listing environment under the moreverb package	175
18.3	Preserving L ^A T _E X syntax in verbatim texts through the alltt environment	175
18.4	Fragile commands in protected mode	176
18.5	Watermarking and logo on pages.	177

18.6	A window within a paragraph	179
18.7	Paragraphs of different shapes under the shapepar package.	180
19	Letter and Article	181
19.1	Standard commands under the document-class letter	181
19.2	Standard format of the document-class letter	182
19.3	A user-specified format in the document-class letter	183
19.4	Article in the document-class article	184
19.5	Article in the document-class amsart	184
19.6	Authors in articles one below another.	185
19.7	Authors side-by-side through the tabular environment.	185
19.8	Author details at the bottom of a page through the \thanks{} command	186
19.9	Article in two columns through the twocolumn option in \documentclass[]{}	187
19.10	Article in two columns through the \twocolumn[] command	188
19.11	Article dividing sections into parts and numbering them part-wise	190
20	Book and Report	191
20.1	General template of a book in the document-class book	192
20.2	User-defined template of a book in the document-class book	193
20.3	Preamble of a book	194
20.4	Some sample input files of a book	195
20.5	Root file linking individual input files of a book	196
20.6	Compilation file for compiling a book in command-prompt	199
20.7	Intermediate files generated during the compilation of a book	200
21	Slide Preparation I	203
21.1	A simple presentation input file	206
21.2	Slides under the JuanLesPins presentation theme for input file of Table 21.1.	207
21.3	Various types of presentation themes under the BEAMER package	210
21.4	Various color themes under the BEAMER package	211
21.5	Various font themes under the BEAMER package	212
21.6	Various inner themes under the BEAMER package	212
21.7	Various outer themes under the BEAMER package	213
21.8	Available frame sizes in the BEAMER package.	215
22	Slide Preparation II	217
22.1	Uncovering slide contents piece-wise using the \pause command	218
22.2	Uncovering items piece-wise using the incremental specification <+>	219
22.3	Slides with incremental overlay specification under the Frankfurt presentation theme	219
22.4	Rules for piece-wise presentation (overlay) specification	220
22.5	Commands which can take overlay specifications.	220
22.6	Applications of some overlay-specification supported commands	221
22.7	Slides with overlay specification under the Hannover presentation theme.	222

22.8	Slides with block-type environments under the Berlin presentation theme	223
22.9	Slides with theorem-like environments under the Warsaw presentation theme	224
22.10	Table in BEAMER presentation	225
22.11	Table in slides with overlay specification under the Madrid presentation theme	226
22.12	Slides with side-by-side materials through the columns environment under the Singapore presentation theme	226
22.13	Hyperlinking slides in BEAMER presentation	228
22.14	Hyperlinked slides under the Boadilla presentation theme	228
Appendix A: Symbols and Notations		247
A.1	Text-mode non-English accents	247
A.2	Text-mode non-English symbols	248
A.3	Text-mode miscellaneous symbols	248
A.4	Greek letters (math-mode)	249
A.5	AMS binary operators (math-mode)	249
A.6	Stmaryrd binary operators (math-mode)	249
A.7	AMS relation operators (math-mode)	250
A.8	Stmaryrd relation operators (math-mode)	250
A.9	AMS negated relation operators (math-mode)	250
A.10	Arrow symbols (math-mode)	251
A.11	AMS arrow symbols (math-mode)	251
A.12	Stmaryrd arrow symbols (math-mode)	251
A.13	Accents and constructs (math-mode)	252
A.14	Delimiter symbols (math-mode)	252
A.15	Mathematical functions (math-mode)	252
A.16	Math-mode miscellaneous symbols	253

List of Figures

	Page
1 Introduction	1
1.1 $\text{\LaTeX}$ and other word processors	1
1.2 Structure of a $\text{\LaTeX}$ input file	2
3 Formatting Texts I	15
3.1 Default three-tier numbering of sectional units	15
4 Formatting Texts II	27
4.1 Increasing depth of sectional units	27
5 Page Layout and Style	37
5.1 Commands controlling some important parameters of a page layout	39
10 Figure Drawing	91
10.1 Coordinate axes for drawing figures	91

Introduction

1.1 What Is L^AT_EX?

Donald E. Knuth developed T_EX in the year 1977 as a typesetting system for preparing books, especially those containing a lot of mathematical expressions. Based on it, Leslie Lamport developed L^AT_EX (named as L^AT_EX 2.09) in 1985 for preparing documents by concentrating on the structure of a document rather than on its formatting details. L^AT_EX 2.09 was enhanced in 1994 as L^AT_EX 2_ε by a group of developers led by Frank Mittelbach.

L^AT_EX is a macro-package used as a language-based approach for typesetting documents. Various L^AT_EX instructions are interspersed with the input file of a document, say `myfile.tex`, for obtaining the desired output as `myfile.dvi` or directly as `myfile.pdf`. The `myfile.dvi` file can be used to generate `myfile.ps` or even `myfile.pdf` file. However, unlike programming languages for computational works, such as C or C++, L^AT_EX is very simple and easy to work with. One can become expert in L^AT_EX through a little practice. L^AT_EX can be used for preparing letters, applications, articles, reports, publications, theses, books, or anything of that kind.

1.2 Why L^AT_EX Over Other Word Processors?

The use of common word processors, in which the effect becomes directly visible, may be easier in preparing simple and small documents. But, as shown roughly in Fig. 1.1, the effort and time required in L^AT_EX for preparing complicated and big-size documents are quite less than those required in other word processors¹. L^AT_EX is especially well suited

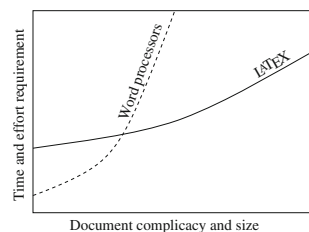


Fig. 1.1 L^AT_EX and other word processors

¹Effort and time required in L^AT_EX for preparing complicated and big-size documents are quite less than those required in other word processors.

for scientific writing, like technical reports, articles, academic dissertations, books, etc. Although learning L^AT_EX is time and effort taking, it can be realized that the preparation of only one academic dissertation would pay off all additional efforts required in learning L^AT_EX.

One of the major advantages of using L^AT_EX is that manual formatting of a document, as usually required in many word processors, can be automated in L^AT_EX. Therefore, the possibility of doing any mistake in numbering and referring items (sections, tables, figures or equations), in choosing size and type of fonts for different sections and subsections, or in preparing bibliographic references, can be avoided. Further, L^AT_EX has the provision for automatically generating various lists of contents, index, and glossary.

1.3 How to Prepare a L^AT_EX Input File?

As shown in Fig. 1.2, the main structure of a L^AT_EX input file can be divided into two parts – *preamble* and *body*.

The preamble is the first part of an input file that contains the global processing parameters for the entire document to be produced, such as the type of the document, page formatting, header and footer setting, inclusion of L^AT_EX packages for supporting additional instructions, and definitions of new instructions. The simplest preamble is `\documentclass{dtype}`, where `dtype` in `{}` is a mandatory argument as the class (or type) of the document, such as **letter**, **article**, **report**, or **book**². In the default setting, `\documentclass{}` prints a document on letter-size paper in 10point fonts (1 point $\approx$ 0.0138 inch $\approx$ 0.3515 mm). Different user-defined formats for a document can be obtained through various options to `\documentclass{}`³, in which case it takes the form of `\documentclass[fo1,fo2,...]{dtype}` with `fo1`, `fo2`, etc., in `[]` as the options (multiple options can be inserted in any order separating two options by a comma), e.g., `\documentclass[a4paper,11pt]{article}` for printing an article on A4 paper in 11 point fonts. Some standard options to `\documentclass[]{}{}` are listed in Table 1.1 on the next page.

As shown in Fig. 1.2, the main body of a L^AT_EX input file starts with `\begin{document}` and ends with `\end{document}`. The entire contents to be printed in the output are inserted within the body, mixed with various L^AT_EX instructions (see §1.5 on page 5 for details). Any text entered after `\end{document}` is simply skipped by a L^AT_EX compiler.

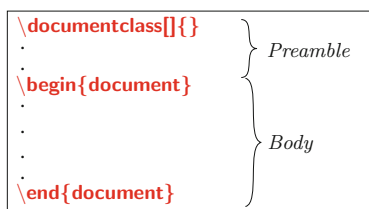


Fig. 1.2 Structure of a L^AT_EX input file

²The standard document classes are **letter**, **article**, **report** and **book**. Besides these four standard classes, some other classes are also available, such as **amsart**, **thesis**, **slide**, **slides** and **seminar**.

³Different user-defined formats for a document can be obtained through various options to `\documentclass[]{}{}`.

Table 1.1 Standard options to the `\documentclass[{}]` command

Format	Options	Function
Font size	10pt (default), 11pt and 12pt	Self-explanatory
paper size	letterpaper (default), a4paper , a5paper , b5paper , legalpaper and executivepaper	Self-explanatory
Page orientation	portrait (default) and landscape	Self-explanatory
Columns of texts	onecolumn (default) and twocolumn	Self-explanatory
Type of printing	oneside (default for article and report) and twoside (default for book)	Self-explanatory
New chapter	openright (default for book) openany	Starts on the next odd page Starts on the next page
Title printing	titlepage and notitlepage	Self-explanatory
Equation	leqno fleqn	Number on left hand side Equations are left aligned
Drafting	final (default) and draft	Self-explanatory
Bibliography	openbib	A part on a separate line

A L^AT_EX input file is named with `tex` extension, say `myfile.tex`. It can be prepared in any operating system using any general-purpose text editor that supports `tex` extension, e.g., `gedit` or `Kate` in Linux-based systems. There are also many open-source (free access in internet) text editors developed specifically for preparing L^AT_EX input files, e.g.,

▷ *For both Windows and Linux:*

- BaKoMa T_EX (<http://bakoma-tex.com>)
- Emacs (www.gnu.org/software/emacs/emacs.html)
- jEdit (<http://jedit.org>)
- Kile (<http://kile.sourceforge.net>)
- LyX (www.lyx.org)
- Open LaTeX Studio (<http://sebburdzinski.github.io/Open-LaTeX-Studio>)
- TeXlipse (<http://texlipse.sourceforge.net>)
- TeXmacs (www.texmacs.org)
- Texmaker (www.xml-math.net/texmaker)
- TeXpen (<https://sourceforge.net/projects/txpen>)
- TeXstudio (www.texstudio.org)
- TeXworks (<https://github.com/TeXworks/texworks>), etc.

▷ *For Linux only:*

- L^AT_EXila (<https://wiki.gnome.org/Apps/LaTeXila>)
- Gummy (<https://github.com/alexandervdm/gummi>), etc.

▷ *For Windows only:*

- Inlage (www.inlage.com)
- LEd (www.latexeditor.org)
- T_EXnicCenter (www.texniccenter.org)
- WinShell (www.winshell.de)
- WinEdt (www.winedt.com), etc.

Table 1.2 A simple L^AT_EX input file and its output

L ^A T _E X input	Output
<pre> \documentclass{article} \begin{document} LaTeX is a macro package for typesetting documents. It is a language-based approach, where LaTeX <i>instructions</i> are interspersed with the text file of a document, say myfile.tex, for obtaining the desired output as myfile.dvi. The myfile.dvi file can then be used to generate myfile.ps or myfile.pdf file. \end{document} </pre>	<u>L^AT_EX</u> is a macro package for typesetting documents. It is a language-based approach, where <u>L^AT_EX instructions</u> are interspersed with the text file of a document, say <u>myfile.tex</u>, for obtaining the desired output as <u>myfile.dvi</u>. The <u>myfile.dvi</u> file can then be used to generate <u>myfile.ps</u> or <u>myfile.pdf</u> file. 1

A simple input file, say `myarticle.tex`, prepared under the document-class of **article** is shown in the left column of Table 1.2, along with its output in the right column. The actual contents to be printed in the output file were inserted within `\begin{document}` and `\end{document}`. Surprisingly, the output is not the one as expected. The differences are shown underlined in the output file. This has happened due to the fact that many things in L^AT_EX can be obtained through some special instructions only as stated in §1.5. Another thing to be noticed is that the output file is assigned a default page number at the bottom-center. However, before going to such issues, the compilation procedure of a L^AT_EX file is stated first in the following section.

1.4 How to Compile a L^AT_EX Input File?

A L^AT_EX input file can be compiled in many L^AT_EX editors mentioned in §1.3 on page 2. Besides, operating-system based many open-source L^AT_EX compilers are also available, e.g., MiKTeX (<http://miktex.org>) for Windows operating system, or TeXLive (<https://www.tug.org/texlive>) for both Windows- and Linux-based operating systems. In a GUI-based compiler, like MiKTeX or Kile, a L^AT_EX file can be compiled just by a mouse-click. In other command-line compilers, a L^AT_EX file is to be compiled through the `latex` command, followed by the name of the input file with or without its `tex` extension. For example, `myarticle.tex` of Table 1.2 can be compiled as follows:

```
$ latex myarticle
```

This command will produce three files, namely `myarticle.aux`, `myarticle.log`, and `myarticle.dvi` (refer §20.4.1 on page 199 for detail). Out of these three files, `myarticle.dvi` is the final output which can be viewed in a document viewer, such as *xdvi* or *Evince*. The `myarticle.dvi` file can also be used for producing `myarticle.ps` or `myarticle.pdf` as the final output of `myarticle.tex`. The commands for these are as follows:

```
$ dvips -o myarticle.ps myarticle.dvi
Or, $ dvi2pdf myarticle.dvi
```

A pdf file can also be generated directly using the `pdflatex` command, instead of the `latex` command as stated above. For example, the following command will also generate `myarticle.pdf`:

```
$ pdflatex myarticle.tex
```

The compilation will need some more commands if a document involves bibliography, lists of contents, index, etc. Such commands will be addressed in relevant Hours.

1.5 L^AT_EX Syntax

L^AT_EX syntax consists of commands and environments, which are kinds of instructions interspersed with the texts of a document for performing some specific jobs. Such instructions are defined in different packages⁴.

1.5.1 Commands

A L^AT_EX command is an independent instruction used either for producing something new or to change the form of an existing item, e.g., producing the symbol α or printing *italic* as *italic*. Different properties of the L^AT_EX commands are stated below:

- ▷ A command usually starts with a `\` (backslash), followed by one or more characters without any gap in between. For example, `\LaTeX` and `\copyright` for producing L^AT_EX and © respectively.
- ▷ Non-alphabetic characters normally cannot appear in the name of a L^AT_EX command. Some L^AT_EX internal commands may start as `\@`, which (commands) are to be put in the preamble preceded and followed, respectively, by the `\makeatletter` and `\makeatother` commands.
- ▷ Many commands require some mandatory arguments, up to a maximum limit of nine, each in a separate pair of `{ }` (preferably without any gap in between), e.g., `\textcolor{blue}{this is blue colored}` (detail is in §2.4 on page 13) for printing the second argument in blue color.
- ▷ Many commands have the provision for accepting some optional instructions also, written in `[ ]` separating two options (instructions) by a comma, e.g., `\documentclass[a4paper,11pt,twoside]{article}`. A command with the provision for optional arguments must have at least one mandatory argument.
- ▷ A command ended by an alphabet (i.e., a command not having any argument) ignores trailing blank spaces. Hence, if followed by a word or a number, such a command should be ended by `\_` (the `_` symbol is used to indicate a blank

⁴In this book, L^AT_EX commands, environments and packages, as well as other L^AT_EX syntax, are printed in **red colored** (for online version) **and boldfaced sans serif fonts** for their clear distinction from other texts of the book.

space obtained by pressing the Spacebar or Tab button of the keyboard). For example, `\copyright 2007` will produce ©2007, while `\copyright 2007` will produce © 2007. However, if such a command is followed by any punctuation, it needs not to be ended by `\` as a punctuation is not to be preceded by any blank space.

1.5.2 Environments

A L^AT_EX environment is a structure composed of two complementary commands, within which some particular job can be performed, e.g., writing an equation or inserting a figure. Different properties of the L^AT_EX environments are as follows:

- ▷ The pair of complementary commands creating an environment structure are `\begin{ename}` and `\end{ename}`, where `ename` is the name of the environment, e.g., `\begin{document}` and `\end{document}` as shown in Fig. 1.2 on page 2 creates the `document` environment (or the body) in a L^AT_EX input file.
- ▷ It is possible to use a command inside an environment, or to nest two or more environments, e.g., within the `document` environment, the `\LaTeX` command for printing L^AT_EX or the `figure` environment for inserting an external figure.
- ▷ Many environments require some mandatory arguments, which are placed after `\begin{}`, e.g., `\begin{spacing}{1.3}` for creating a line spacing of 1.3 pt through the `spacing` environment, or `\begin{tabularx}{10cm}{XXX}` for creating a table of three equal-width columns over 10 cm length through the `tabularx` environment.
- ▷ Like a command, many environments also have the provision for accepting some optional instructions written in a pair of `[ ]`, e.g., `\begin{table}[t]` preferring through the option `t` in the `table` environment to place a table at the top of a page.

1.5.3 Packages

The class (or type) of a document, incorporated through the mandatory argument of the `\documentclass{}` command, includes some basic features of the document, like page layout and sectioning. Provision is also there to invoke additional commands and environments in a document for adding extra features that are not parts of the standard document class. Such commands and environments are defined in separate files, known as packages.

- ▷ A package is loaded (included) in the preamble, in between the `\documentclass{}` and `\begin{document}` commands⁵, through the `\usepackage{pname}` command, where `pname` is the name of the package, e.g., `\usepackage{color}` for producing colored texts or `\usepackage{amssymb,amsmath}` for producing AMS type mathematical symbols and expressions.

⁵If anything (a command or a package) is asked to be put in the preamble of a L^AT_EX input file, it is to be inserted in between the `\documentclass{}` and `\begin{document}` commands.

- ▷ Like commands and environments, many packages also have the provision for accepting some optional instructions in `[]`, e.g., `\usepackage[tight]{subfigure}` preferring through the option **tight** to reduce extra space between figures.
- ▷ Unlike an option to `\documentclass[]`, which is global to the entire document – including other packages too, an option to `\usepackage[]` is local only to the features defined in the package(s) loaded through the `\usepackage[]` command.

1.6 Keyboard Characters in L^AT_EX

Not all, but only those characters of an *English keyboard*⁶, shown in Table 1.3, can be

Table 1.3 Keyboard characters that can be produced directly

Type of character	Characters
Uppercase letters	A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
Lowercase letters	a b c d e f g h i j k l m n o p q r s t u v w x y z
Digits	0 1 2 3 4 5 6 7 8 9
Parentheses	()
Brackets	[]
Quotations	` ' " ~
Punctuation	, ; : ! . ?
Math operators	+ - * / =
Other symbols	@

printed directly in a L^AT_EX document (the left-hand quotation mark (`), shown in Table 1.3, generally appears on the same button with the ~ symbol). All other characters of an English keyboard, which are not included in Table 1.3, need to be produced in a L^AT_EX document through some commands (most of these characters are reserved in L^AT_EX for special purposes). Table 1.4 on the following page lists those special characters, commands for producing them in a L^AT_EX document, and also the purposes, if any, for which these are reserved in L^AT_EX. For the commands starting and ending with the \$ symbol (i.e., in \$ \$), the **amssymb** package may be required (when used in an equation, as addressed in Hour 11 on page 101, these commands need not to be enclosed in \$ \$).

The special keyboard characters, listed in Table 1.4, can be produced in text-mode using the `\verb" "` or `\verb! !` command also, e.g., `\verb"$"` for printing \$, or `\verb!~!` for printing ~ (the `\verb" "` and `\verb! !` commands are used for printing *as-it-is* what is written within " " or ! ! in the input file).

The commands for producing language-specific keyboard characters, as well as many other characters and mathematical symbols, are given in Appendix A on page 247.

⁶Many computers are manufactured for particular countries where a keyboard contains some language-specific characters, in addition to those used in the English language. However, for general purpose, a keyboard containing the characters, used in the English language only, will be discussed in this book.

Table 1.4 Keyboard characters to be produced through commands

Character	Command	Function in L ^A T _E X
\$	\\$	A pair of \$ creates a math-mode* within text-mode.
%	\%	Texts of a line preceded by % are commented.
{ }	\{ \}	Mandatory arguments of a command are written within {}.
_	_	Generates a subscript in math-modes.
^	\^	Generates a superscript in math-modes.
&	\&	Separates the entries of two columns in a Table.
#	\#	Miscellaneous symbol.
\	\backslash	Most of the L ^A T _E X commands start with \.
~	\sim	Binds two words to be printed in the same line.
	\$ 	Generates a vertical (column) line in a Table.
<	\$<	—
>	\$>	—

*Text processing modes are discussed in the first paragraph of Hour 2 on the next page.

1.7 How to Read this Book?

The version L^AT_EX 2_ε of L^AT_EX is discussed in this book. However, without referring the version, only the general term L^AT_EX is used throughout the book.

It is not a reference manual, but a practical guide prepared based only on the author’s own experiences. The book is intended for beginners, and hence it discusses mainly the basic elements available in the standard L^AT_EX 2_ε distribution. Many relevant advanced topics are also discussed marking them as starred (*), which can be skipped in the initial stage of learning. Many statements are repeated in the book so that a reader is not necessarily required to remember everything. All L^AT_EX syntax, including commands and environments, are printed in **red colored** (for online version) **and boldfaced sans serif fonts** to make them easily distinguishable from normal texts.

Since the book is meant for beginners, it is suggested to read the entire book for better understanding of L^AT_EX. However, as mentioned in the Preface that many students and professionals are interested to get their works done in less time and least efforts, one can move directly to Hour 19 on page 181 or Hour 20 on page 191 to start writing a document immediately. Then other Hours can be followed for required additional information, where different topics – writing equations, drawing tables, inserting figures, etc., are discussed in detail. Also, many important points are highlighted in foot notes on various pages. For quickly locating the availability of different topics, one may browse through the Contents, List of Tables and List of Figures at the beginning of the book, or Index at the end of the book for searching different terminologies, where attempts are made to include information almost about all the materials that are discussed in this book.

Fonts Selection

There are three modes for processing texts in L^AT_EX – *paragraph*-mode, *LR*-mode and *math*-mode¹. The paragraph-mode is for producing normal texts with automatic word-splitting, and line and page breaking to fit the texts within the area specified by the width and height of a page. In contrast, the LR-mode processes texts from left-to-right without any word-splitting and line breaking, such as `\mbox{}` or `\vbox{}` command whose arguments may span even beyond the specified width of a page. On the other hand, the math-mode is for writing mathematical expressions, like equations. In this book, the paragraph-mode and LR-mode will occasionally be addressed by a single name, known as the *text*-mode.

2.1 Text-Mode Fonts

The default font type of a L^AT_EX document is medium series serif family in upright shape and 10pt size. The sizes of fonts in different parts of a document, say in headings and in paragraphs, are calculated proportionately, which can be visualized in this book. The default font setting can be altered globally through various options to the `\documentclass[]{}` command, e.g., `\documentclass[12pt]{article}` for producing an article in 12pt fonts. The type of fonts in a particular segment can also be set manually as discussed below.

Types of fonts in L^AT_EX are classified into four categories – *family*, *series*, *shape* and *size*. The detail of each category is given in Table 2.1 on the following page.

1. *Font family*: There are three standard font families, namely *serif* (default), *sans serif* and *typewriter* fonts, which are accessed by the `\textrm{}` (or `\rm`), `\textsf{}` (or `\sf`) and `\texttt{}` (or `\tt`) commands respectively. The same can also be accessed by the `\rmfamily`, `\sffamily` and `\ttfamily` declarations, respectively.

¹L^AT_EX processes texts in three modes – paragraph-mode, LR-mode and math-mode.

Table 2.1 Different types of text-mode fonts used in L^AT_EX

SN	Type	Variety	Command	Declaration
1	Family	Serif family (default)	<code>\textrm{atext}</code> or <code>{\rm atext}</code>	<code>\rmfamily</code>
		Sans serif family	<code>\textsf{atext}</code> or <code>{\sf atext}</code>	<code>\sffamily</code>
		Typewriter family	<code>\texttt{atext}</code> or <code>{\tt atext}</code>	<code>\ttfamily</code>
2	Series	Medium series (default)	<code>\textmd{atext}</code>	<code>\mdseries</code>
		Boldface series	<code>\textbf{atext}</code> or <code>{\bf atext}</code>	<code>\bfseries</code>
3	Shape	Upright shape (default)	<code>\textup{atext}</code>	<code>\upshape</code>
		<i>Italic shape</i>	<code>\textit{atext}</code> or <code>{\it atext}</code>	<code>\itshape</code>
		<i>Slanted shape</i>	<code>\textsl{atext}</code> or <code>{\sl atext}</code>	<code>\slshape</code>
		CAPS & SMALL CAPS SHAPE	<code>\textsc{atext}</code> or <code>{\sc atext}</code>	<code>\scshape</code>
		<i>Emphasized shape</i>	<code>\emph{atext}</code> or <code>{\em atext}</code>	—
4	Size	Tiny size	<code>{\tiny atext}</code>	<code>\tiny</code>
		Script size	<code>{\scriptsize atext}</code>	<code>\scriptsize</code>
		Foot note size	<code>{\footnotesize atext}</code>	<code>\footnotesize</code>
		Small size	<code>{\small atext}</code>	<code>\small</code>
		Normal size (default)	—	<code>\normalsize</code>
		Large size	<code>{\large atext}</code>	<code>\large</code>
		Larger size	<code>{\Large atext}</code>	<code>\Large</code>
		Largest size	<code>{\LARGE atext}</code>	<code>\LARGE</code>
		Huge size	<code>{\huge atext}</code>	<code>\huge</code>
		Hugest size	<code>{\Huge atext}</code>	<code>\Huge</code>

2. *Font series*: The two series of fonts, medium-valued width and height (default) and boldface, are accessed respectively by the `\textmd{}` and `\textbf{}` (or `{\bf }`) commands (or corresponding `\mdseries` and `\bfseries` declarations).
3. *Font shape*: Fonts of four different shapes, upright (default), italic, slanted, and caps and small caps, can be produced respectively through the `\textup{}`, `\textit{}` (or `{\it }`), `\textsl{}` (or `{\sl }`) and `\textsc{}` (or `{\sc }`) commands (or corresponding `\upshape`, `\itshape`, `\slshape` and `\scshape` declarations). Apart from these four shapes, fonts of emphasized shape can be produced using the `\emph{}` or `{\em }` command.
4. *Font size*: Fonts of ten different sizes can be produced using the `{\tiny }`, `{\scriptsize }`, `{\footnotesize }`, `{\small }`, `{\normalsize }`, `{\large }`, `{\Large }`, `{\LARGE }`, `{\huge }` and `{\Huge }` commands (or their corresponding declarations of `\tiny`, `\scriptsize`, `\footnotesize`, `\small`, `\normalsize`, `\large`, `\Large`, `\LARGE`, `\huge` and `\Huge` respectively). The sizes of these ten types of fonts are not rigid, but proportional to the setting made in the document class, e.g., `\documentclass[12pt]{}` for producing fonts of 12 pt in size.

Notice that italic, emphasized and slanted letters are leaned towards right, for which the gap between the last italic, emphasized, or slanted letter and the following upright letter gets reduced. In order to maintain a proper spacing, the arguments of the `{\it }`, `{\em }` and `{\sl }` commands may be followed by `\V`. For example, `{\it red} line`

will produce ‘*red line*’, while ‘`\it red\` line’ will produce ‘*red line*’. The `\textit{}`, `\emph{}` and `\textsl{}` commands make such corrections automatically. Further, the `\` symbol is not required if the last italic, emphasized, or slanted letter is followed by a punctuation. The `\` can also be used between two letters for increasing inter-letter spacing, e.g., ‘of`\fV\`ice’ will produce ‘office’, while ‘office’ produces ‘office’.

Different combinations of font family, series, shape and size (i.e., the commands of Table 2.1) in a logical way are allowed for producing a wide variety of fonts², e.g., `\emph{\textbf{emphasized boldface fonts}}` for producing ‘***emphasized boldface fonts***’, or `\large\sf large sans serif fonts` for producing ‘large sans serif fonts’.

Type of fonts of an individual word or a short phrase can be changed by a font command having an argument, e.g., `\textbf{atext}` for printing `atext` in boldface fonts. While an equivalent declaration without any argument, e.g., `\bfseries`, may be used for changing the fonts of a large portion, say the remaining texts of an environment or a document. To return to the main document fonts, the declaration of any specific font type can be quit using the `\normalfont` declaration. If a particular font type is to be used in one or more consecutive paragraphs, the font type can be applied as an environment also, e.g., `\begin{bfseries}aparas\end{bfseries}` for printing `aparas` in boldface fonts.

2.2 Math-Mode Fonts

Like in text-mode, different types of fonts can be used in math-mode also as shown in Table 2.2 (math-mode is discussed in Hour 11 on page 101). In the case of these math-mode fonts, the following three points are to be noted:

Table 2.2 Different types of math-mode fonts used in L^AT_EX

Font type	Command	Package required	Output
Serif family	<code>\mathrm{ABC abc}</code>	—	ABCabc
Italic shape	<code>\mathit{ABC abc}</code>	—	<i>ABCabc</i>
Boldface series	<code>\mathbf{ABC abc}</code>	—	ABCabc
Sans serif family	<code>\mathsf{ABC abc}</code>	—	ABCabc
Typewriter family	<code>\mathtt{ABC abc}</code>	—	ABCabc
Mathematical boldface	<code>\boldmath{ABC abc}</code>	amssymb	<i>ABCabc</i>
Mathematical normal	<code>\mathnormal{ABC abc}</code>	—	<i>ABCabc</i>
Calligraphic	<code>\mathcal{A B C}</code>	—	<i>A B C</i>
Open	<code>\Bbb{A B C}</code>	amsfonts/ amssymb	A B C
Open	<code>\mathbb{A B C}</code>	amsfonts/ amssymb	A B C
German/ Fraktur	<code>\mathfrak{ABC abc}</code>	eufrak/ amsfonts/ amssymb	A B Cabc

²Different combinations of font family, series, shape and size (i.e., the commands of Table 2.1) in a logical way are allowed for producing a wide variety of fonts.

1. If used in text-mode, the commands of Table 2.2 (except `\boldmath{}`) are to be written within a pair of `$` symbol, e.g., `$\mathbf{abc}$` for printing **abc**. In the case of the `\boldmath{}` command, the argument is to be enclosed in a pair of `$` symbol, e.g., `\boldmath{$abc$}` for printing **abc**.
2. The `\mathcal{}`, `\mathbb{}` and `\Bbb{}` commands do not accept lower case letters.
3. Any blank space in the arguments of the commands of Table 2.2 is omitted. Commands, like `\,` or `\sim`, may be used for maintaining some gap between two letters or words, e.g., `$\mathbb{A}\,\mathbb{B}\sim\mathbb{C}$` will produce $\mathbb{A} \mathbb{B} \mathbb{C}$ (creating blank spaces is discussed in §3.6 on page 21, while texts in math-mode in §12.1 on page 113). However, most of the commands of different family, series, and shape having the forms of `\text{. . .}` (e.g., `\textbf{}` or `\textit{}`) and `\emph{}`, as shown in Table 2.1, can be used for writing normal texts in math-mode preserving the space provided between two letters in the input file.

2.3 Emphasized Fonts

Important texts in a document are usually emphasized by writing them in **boldface**, *italic*, or in **boldface italic**, which are done in L^AT_EX through the `\bf`, `\it` (or `\em`) and `\bfit` commands respectively, or through their other forms of `\textbf{}`, `\textit{}` (or `\emph{}`) and `\textbf{\emph{}}` commands³ respectively (refer §2.1 for detail).

Apart from the above forms, texts can also be emphasized by underlining them through the `\underline{}` command, e.g., `\underline{important}` will produce important. However, the `\underline{}` command does not permit any line break in between its argument, for which it cannot be used for printing a long statement as it may go beyond the margin of a line.

The above problem with the `\underline{}` command can be sorted out using the `ulem` package, which redefines the `\em` and `\emph{}` commands for printing their arguments by underlining with required line breaks. If some texts are to be underlined, as well as to be printed in *italic* fonts, either `\it\em` or `\textit{\emph{}}` command may be used. If the `ulem` package is loaded, the redefined effects of the `\em` and `\emph{}` commands can be turned on or off using the `\ULforem` or `\normalem` command in between the texts where from the effects are to be turned on or off respectively. Besides the `\em` and `\emph{}` commands, the `ulem` package provides the `\uwave{}`, `\sout{}` and `\xout{}` commands for printing an argument, respectively, by a wavy underline, a strike-out line and by crossing out each character of the argument⁴.

³Both the `\emph{}` and `\em` commands produce emphasized fonts, but the `\emph{}` command produces better spacing than the `\em` command does.

⁴Struck out or overlapping texts can also be produced by creating a negative horizontal space between two pieces of texts using the `\hspace{}` command with a negative length as its argument, e.g., `'struck out'` can be produced by `'struck out\hspace{-1.8cm}'`.

Some examples of various features of the `ulem` package are shown in Table 2.3, where different applications are numbered both in L^AT_EX input and out files for easily identifying the effect of each L^AT_EX syntax. A blank line is left in the input file before each application for printing it in a new line (creating new lines is discussed in §3.5.1 on page 19), while the texts in a line preceded by a `%` sign are simply commented. The underlining effects of the redefined `\em` and `\emph` commands are shown in applications 2–5, where the redefined effects are first switched on by putting the `\ULforem` command before application 2 and then they are switched off by putting the `\normalem` command after application 5. Note that the `\uwave{}`, `\sout{}` and `\xout{}` commands are not affected by the `\normalem` and `\ULforem` commands.

Table 2.3 Various forms of emphasized texts under the `ulem` package

L ^A T _E X input	Output
<pre>\documentclass[11pt,a4paper]{article} \usepackage{ulem} \begin{document} 1. {\em Normal emphasized texts.} \ULforem % Redefining effects of {\em } and \emph{.} 2. {\em Underlined texts with line breaks.} 3. \emph{Yet underlined texts with line breaks.} 4. {\it{\em Underlined texts in italic fonts ...}} 5. \textit{\emph{Yet underlined texts in ...}} \normalem % For normal effects of {\em } and \emph{.} 6. {\em Returned to normal emphasized texts.} 7. The next phase is \uwave{wavy underlined.} 8. The next phase is \sout{striked out statement.} 9. The next phase is \xout{crossed out statement.} \end{document}</pre>	1. <i>Normal emphasized texts.</i> 2. <u>Underlined texts with line breaks.</u> 3. <u>Yet underlined texts with line breaks.</u> 4. <i><u>Underlined texts in italic fonts with necessary line breaks.</u></i> 5. <i><u>Yet underlined texts in italic fonts with necessary line breaks.</u></i> 6. Returned to normal emphasized texts. 7. The next phase is <u>wavy underlined.</u> 8. The next phase is striked out statement. 9. The next phase is crossed out statement.

2.4 Colored Fonts

Like many word-processors, L^AT_EX also has the provision for producing colored fonts, supported by the `color` package. There are basically three types of color combinations – black and white (`gray`), additive primaries (`rgb`) and subtractive primaries

(**cm**yk)⁵. Using the `\definecolor{c}{gray}{w}` command, various colors can be defined by setting different values to **gray** and each of the letters of **rgb** and **cm**yk as follows:

```
\definecolor{c}{gray}{w}           ; w ∈ [0, 1]
\definecolor{c}{rgb}{w, x, y}      ; w, x, y ∈ [0, 1]
\definecolor{c}{cm}{y}{w, x, y, z} ; w, x, y, z ∈ [0, 1]
```

The definitions of the predefined colors, as well as some examples of defining new colors, are shown in Table 2.4, where `cname` is the name of the user-defined new color. The predefined colors (**black**, **white**, **red**, **green**, **blue**, **cyan**, **magenta** and **yellow**) need not to be redefined, while the user-defined colors (as shown through an example in Table 2.4 under each category of color combination) may be defined in the preamble for global effect or inside the **document** environment for local effect.

Table 2.4 Predefined as well as some user-defined colors for fonts

Type	Command	Color
Black and white	<code>\definecolor{black}{gray}{0}</code>	Predefined black
	<code>\definecolor{white}{gray}{1}</code>	Predefined white
	<code>\definecolor{c}{gray}{0.75}</code>	User-defined
Additive primaries	<code>\definecolor{red}{rgb}{1, 0, 0}</code>	Predefined red
	<code>\definecolor{green}{rgb}{0, 1, 0}</code>	Predefined green
	<code>\definecolor{blue}{rgb}{0, 0, 1}</code>	Predefined blue
	<code>\definecolor{black}{rgb}{0, 0, 0}</code>	Predefined black
	<code>\definecolor{white}{rgb}{1, 1, 1}</code>	Predefined white
	<code>\definecolor{c}{rgb}{0, 0.7, 0.3}</code>	User-defined
Subtractive primaries	<code>\definecolor{cyan}{cm}{y}{1, 0, 0, 0}</code>	Predefined cyan
	<code>\definecolor{magenta}{cm}{y}{0, 1, 0, 0}</code>	Predefined magenta
	<code>\definecolor{yellow}{cm}{y}{0, 0, 1, 0}</code>	Predefined yellow
	<code>\definecolor{black}{cm}{y}{1, 1, 1, 1}</code>	Predefined black
	<code>\definecolor{white}{cm}{y}{0, 0, 0, 0}</code>	Predefined white
	<code>\definecolor{c}{cm}{y}{0.2, 1, 0.7, 0}</code>	User-defined

Once different colors are defined as above (if required), colored texts can be produced through the `\textcolor{c}{atext}` command, where `atext` is the piece of texts to be colored by `cname` color. For example, `\textcolor{blue}{this is in blue}` will print ‘this is in blue’, while `\textcolor{ur}{this is in rgb = {0, 0.7, 0.3}}` will print ‘this is in rgb = {0, 0.7, 0.3}’, where `ur` is a new color defined as `\definecolor{ur}{rgb}{0, 0.7, 0.3}` (recall that red color is adopted in this book for writing L^AT_EX commands and other L^AT_EX syntax).

The fonts discussed in §2.1–§2.3 can also be colored through the `\textcolor{c}{}` command, e.g., `\textcolor{magenta}{\small\sf small Sans serif in magenta}` will produce ‘small Sans serif in magenta’, and `\textcolor{blue}{\mathfrak{Colored~Fraktur~fonts}}` will produce ‘Colored Fraktur fonts’.

⁵There are basically three types of color combinations – black and white (**gray**), additive primaries (**rgb**) and subtractive primaries (**cm**yk).

Formatting Texts I

Although L^AT_EX has numerous predefined macros for automatic and uniform formatting of a document without any mistake¹, many L^AT_EX users commit mistake by attempting to format a document manually. Manual formatting includes inserting vertical or horizontal spacing with `\bigskip`, `\vskip`, `\vfill`, etc., setting sectional headings via explicit font instructions, such as `\centerline{\bf Section 1: Introduction}` or `{\bf Theorem 4:}`, etc. Another mistake may be committed in keeping track of manually set sectional units. Various options for automatic formatting of a document are discussed here.

3.1 Sectional Units

Various sectional units, like chapters and sections, are generated using the `\chapter{}`, `\section{}`, `\subsection{}`, `\subsubsection{}`, `\paragraph{}` and `\subparagraph{}` commands, whose argument is the heading or title of a sectional unit, e.g., the current section of this book is written as `\section{Sectional Units}`. The sectional unit commands work in order and hence they should be nested properly, i.e., a `\subsection{}` command should follow a `\section{}` command or a `\subparagraph{}` command should follow a `\paragraph{}` command. L^AT_EX assigns three-tier serial numbers to chapters, sections, subsections, and subsubsections (paragraphs and subparagraphs are not numbered). These numbering system is shown in Fig. 3.1.

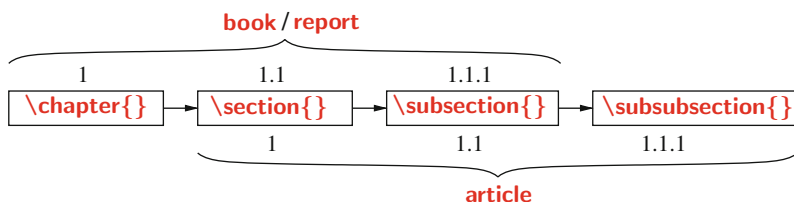


Fig. 3.1 Default three-tier numbering of sectional units

¹L^AT_EX has numerous predefined macros for automatic and uniform formatting without any mistake.

In the case of document-class **report** or **book**, which is composed of chapters, numbering is possible only to the `\chapter{}`, `\section{}`, and `\subsection{}` commands. Even if a `\subsubsection{}` command is used, it will not be numbered in the document-class **report** and **book**. A chapter is numbered by a whole number preceded by the label-word ‘Chapter’ and followed by its heading. A chapter may contain a number of sections, which are designated as 1.2, 3.5, etc., where the second number is the serial number of a section and the first number is the serial number of the chapter in which the section belongs. Similarly, the third tier of serial numbers are assigned to subsections, which are numbered as 2.5.3, 6.2.2, etc. On the other hand, since the document-class **article** is composed of sections and paragraphs only (it does not support the `\chapter{}` command), the three-tier numbering is assigned to the `\section{}`, `\subsection{}` and `\subsubsection{}` commands, whose numbering is similar with the numbering of `\chapter{}`, `\section{}` and `\subsection{}` commands in the document-class **report** and **book** (however, the first numbering is not preceded by any label-word as done by ‘Chapter’ in **report** and **book**). Note that the document-class **letter** does not support any sectional unit.

In some cases, the numbering of a sectional unit may need to be omitted (e.g., the *Preface* of a book is usually written under the `\chapter{}` command, but it is not numbered). L^AT_EX takes care of it through its `\chapter*{}`, `\section*{}`, `\subsection*{}` and `\subsubsection*{}` commands, which are not numbered because of their starred (*) forms.

3.2 Labeling and Referring Numbered Items

Like to sectional units addressed in §3.1, L^AT_EX assigns serial numbers to many environments or elements of an environment (e.g., **table**, **figure**, **equation**, or **item** as discussed in following Hours). This default numbering system eliminates the possibility of committing any mistake as may happen in manual numbering. Moreover, L^AT_EX allows to label a numbered item by a unique *reference key*, which can be used to refer the item in any part within the same document (unnumbered items, say `\paragraph{}`, cannot be referred in this way). The labeling and referring of an item are performed through `\label{rkey}` and `\ref{rkey}` respectively, where *rkey* is the assigned unique reference key of the item².

An example of labeling and referring an item is shown in Table 3.1 on the next page. The `\section{}` commands are followed by the `\label{}` commands containing the reference keys of ‘sec:cg’ and ‘sec-ex’ respectively (two different types of reference keys are considered to show that labeling can be done in any fashion, but without any blank space in between the texts of a reference key). In the body of the second `\section{}`, the first `\section{}` is referred through the `\ref{}` command using

²A numbered item can be labeled using `\label{rkey}` and then it can be referred using `\ref{rkey}` anywhere within the same document, where *rkey* is the assigned unique reference key of the item.

Table 3.1 Labeling and referring numbered items

L ^A T _E X input	Output
<pre>\section{Centre of gravity}\label{sec:cg} This is the point though which the resultant of the gravitational forces of all elemental weights of a body acts. % \section{Centre of mass}\label{sec-ex} The definition of the centre of gravity is given in Section~\vref{sec:cg} ...</pre>	3.2 Centre of gravity This is the point though which the resultant of the gravitational forces of all elemental weights of a body acts. 3.3 Centre of mass The definition of the centre of gravity is given in Section 3.2 ...

its reference key as the argument of the command. As a result, the first section is referred in the output automatically by its serial number (not by its reference key), thus eliminates any possibility of referring an item by a wrong number.

Similar to the `\ref{}` command, there exists some commands for printing the page number where a numbered item appears. Such commands include `\pageref{rkey}`, `\vref{rkey}` and `\vpageref{rkey}`³.

- ▷ The `\pageref{}` command prints the page number where its argument is labeled, e.g., ‘CG is defined on page~`\pageref{sec:cg}`’ will print ‘CG is defined on page 17’. The `\pageref{}` command may not be preferred, particularly when labeling and referring of an item appear on the same page, as the command will print the serial number of a page on that page itself as done in the above example. Instead of generating the page number, some other texts, such as ‘on this page’ may suit better in this situation. Such facilities are provided in the `\vref{}` and `\vpageref{}` commands, which are defined in the `varioref` package.
- ▷ The `\vref{}` command usually executes a pair of `\ref{}` and `\pageref{}` commands.
 - It executes only the `\ref{}` command when labeling and referring of an item appear on the same page.
 - In addition to executing the `\ref{}` command, the `\vref{}` command also generates strings like ‘on the facing page’, ‘on the preceding page’ or ‘on the next page’ when the page numbers of labeling and referring of an item differ by one.
 - When the difference in page numbers of labeling and referring of an item is more than one, the `\vref{}` command executes both the `\ref{}` and `\pageref{}` commands. For example, ‘CG is defined in `\S\vref{sec:cg}`’ will print ‘CG is defined in §3.2’, while ‘quoted texts are discussed in `\S\vref{sec:quote}`’ will print ‘quoted texts are discussed in §3.4 on the next page’, or ‘text-mode fonts are discussed in `\S\vref{sec:text-fonts}`’ will print ‘text-mode fonts are discussed in §2.1 on page 9’.

³The `\pageref{rkey}`, `\vref{rkey}` and `\vpageref{rkey}` commands may be used in a similar way to `\ref{}` for referring the page number of an item, whose assigned unique reference key is `rkey`.

▷ If the `\ref{}` command is to be ignored, but strings similar to that produced by the `\vref{}` command are required, the `\vpageref{}` command may be used. For example, ‘CG is defined `\vpageref{sec:cg}`’ will print ‘CG is defined on the preceding page’. Two more controls can be obtained through two optional arguments to the `\vpageref{}` command as `\vpageref[asame][adiffer_]{rkey}`, where `asame` specifies the texts to be printed when labeling and referring appear on the same page, while `adiffer` specifies the texts if they fall on different pages. For example, ‘the `\vpageref[above definition][definition_]{sec:cg}` is for the center of gravity’ will produce ‘the definition on the preceding page is for the center of gravity’. Notice that a blank space (`_`) is maintained after `adiffer` in `\vpageref[asame][adiffer_]{rkey}`, i.e., after the string of the second optional argument, for maintaining a gap between the string and the page number (this is not required for the first argument, in which case no page number is generated).

3.3 Texts Alignment

By default `LATEX` prints texts with both side aligned. Other types of alignment can be obtained through the `flushleft`, `flushright` and `center` environments, which print texts left, right, and center aligned, respectively. Some applications of these environments are shown in Table 3.2. Such alignments can also be produced using the corresponding declarations of these environments, which are `\raggedright`, `\raggedleft` and `\centering`, respectively.

Table 3.2 User-defined alignments of texts

L ^A T _E X input	Output
<code>\begin{flushleft}</code> <code>\LaTeX</code> prints texts with both side aligned. Left aligned texts can be produced through the ‘flushleft’ environment. <code>\end{flushleft}</code>	<code>L^AT_EX</code> prints texts with both side aligned. Left aligned texts can be produced through the ‘flushleft’ environment.
<code>\begin{flushright}</code> <code>\LaTeX</code> prints texts with both side aligned. Right aligned texts can ... <code>\end{flushright}</code>	<code>L^AT_EX</code> prints texts with both side aligned. Right aligned texts can be produced through the ‘flushright’ environment.
<code>\begin{center}</code> <code>\LaTeX</code> prints texts with both side aligned. Center aligned texts can ... <code>\end{center}</code>	<code>L^AT_EX</code> prints texts with both side aligned. Center aligned texts can be produced through the ‘center’ environment.

3.4 Quoted Texts

Often some texts are required to be quoted within single or double quotation marks, which are usually done using `( ’ )` and `( " )` symbols (generally both appear on the same button of a keyboard). However, `LATEX` suggests to use `( ’ )` as the left-hand

single quote and (') as the right-hand single quote (‘ generally appears in a keyboard on the same button with ~). For double quotation mark, the single quote symbols may be used twice. For example, ‘single-quote’ will produce ‘single-quote’, while “double-quote” will produce “double-quote”.

Sometime an existing statement may need to be quoted without any change, which is usually done in a narrowed width of a page. L^AT_EX provides the **quote** and **quotation** environments for that purpose. Both the environments are used in the same way, but the **quote** is usually for a short display, while the **quotation** for quoting more than one paragraph. Quoted statements can also be printed in a different line spacing through the **spacing** environment. An application of the **quotation** and **spacing** environments is shown in Table 3.3, where the mandatory argument of the **spacing** environment (1.2 is taken here) specifies the line spacing to be maintained within the

Table 3.3 Quoted texts in a narrowed width and specified line spacing

L ^A T _E X input	Output
<pre>\LaTeX\ prints texts with both side aligned, covering the specified width of a page. \begin{quotation} \begin{spacing}{1.2} Quoted statements are also printed with both side aligned, but in a narrowed width. \begin{flushright} {\it - Anonymous} \end{flushright} \end{spacing} \end{quotation} The 'quotation' environment is used for printing quoted statements in a narrowed width.</pre>	L^AT_EX prints texts with both side aligned, covering the specified width of a page. Quoted statements are also printed with both side aligned, but in a narrowed width. - Anonymous The 'quotation' environment is used for printing quoted statements in a narrowed width.

environment. If single line spacing is required, instead of the **spacing** environment, the **singlespace** environment may also be used without any argument. Both the **spacing** and **singlespace** environments are supported by the **setspace** package. Notice the nested environments in Table 3.3, where three environments, **quotation**, **spacing** and **flushright**, are nested. It is permitted in L^AT_EX to create such nested environments.

3.5 New Lines and Paragraphs

L^AT_EX does not respond to a new line or paragraph set manually by pressing the enter button of the keyboard. Unless specified commands are used, L^AT_EX considers everything in a single line and single paragraph.

3.5.1 Creating New Lines

The direct command for creating a new line is **\newline**. The texts preceded by a **\newline** command are printed in a new line. A new line can also be created using a line break command (**\linebreak**, **\l**, **\lll**, or one or more blank lines) at the end of

the previous line⁴. Some applications of these commands, using the same sentence, are shown in Table 3.4. To easily distinguish the differences of the commands, the

Table 3.4 Creating new lines

LaTeX input	Output
(1) There are certain key issues to attract investors, which need to be addressed. \newline	(1) There are certain key issues to attract investors, which need to be addressed.
(2) There are certain key issues to attract investors, which need to be addressed. \linebreak	(2) There are certain key issues to attract investors, which need to be addressed.
(3) There are certain key issues to attract investors, which need to be addressed. \	(3) There are certain key issues to attract investors, which need to be addressed.
(4) There are certain key issues to attract investors, which need to be addressed. \	(4) There are certain key issues to attract investors, which need to be addressed.
(5) There are certain key issues to attract investors, which need to be addressed.	(5) There are certain key issues to attract investors, which need to be addressed.
(6) There are certain key issues to attract investors, which need to be addressed.	(6) There are certain key issues to attract investors, which need to be addressed.

sentence is numbered in each case. The effects of **\linebreak** in (2), **** in (4), and a blank line prior to (6), are most noticeable. The **\linebreak** command forces the texts of the last line to cover the entire page width. The **** command not only prints the following texts in a new line, but creates a blank line also. On the other hand, a blank line in the input file creates a new line with indentation (some blank space) at the beginning of the line (same effect would be resulted even if more than one blank line were left).

Some extra vertical space above the next new line can be specified in **[]** after the **** command at the end of the previous line, e.g., **\[2mm]** will create an extra vertical space of 2 mm above the next line. Further, the ***** or ***[]** command may be used for line breaking, but preventing page breaking at that point.

3.5.2 Creating New Paragraphs

Although a new paragraph can be started manually by creating a new line as discussed in §3.5.1, LaTeX suggests to use its predefined macros for creating uniform paragraphs. The direct command for creating a new paragraph is **\par**. On the other hand, the **\paragraph{ }** and **\subparagraph{ }** commands can also be used for creating new paragraphs with the arguments of the commands as the headings of the paragraphs. Table 3.5 on the facing page shows some applications of these commands, along with three more commands **\parindent**, **\noindent** and **\parskip**. The first **\par** command prints the texts in (1) in a new paragraph with default indentation. The indentation size for paragraph (2) is increased to 8 mm through the **\parindent = 8mm** command, while the indentation for paragraph (3) is skipped using the **\noindent**

⁴LaTeX does not create a new line set manually by pressing the enter button of the keyboard, rather the same can be obtained by inserting preferably **\newline** or **** at the end of the previous line.

Table 3.5 Creating new paragraphs

LaTeX input	Output
<pre>\par (1) There are certain key issues to attract investors, which need to be addressed. % \par \parindent = 8mm (2) There are certain key issues to attract investors, which need to be addressed. % \par \noindent \parskip 2mm (3) There are certain key issues to attract investors, which need to be addressed. \par (4) There are certain key issues to attract investors, which need to be addressed. % \paragraph{(5) Investing policies:} There are certain key issues to attract investors, which need to be addressed. % \subparagraph{(6) Investing policies:} There are certain key issues to attract investors, which need to be addressed.</pre>	(1) There are certain key issues to attract investors, which need to be addressed. (2) There are certain key issues to attract investors, which need to be addressed. (3) There are certain key issues to attract investors, which need to be addressed. (4) There are certain key issues to attract investors, which need to be addressed. (5) Investing policies: There are certain key issues to attract investors, which need to be addressed. (6) Investing policies: There are certain key issues to attract investors, which need to be addressed.

command (indentation can be reactivated using the `\indent` command). Further, a vertical space of 2 mm is created above paragraph (3) using the `'\parskip 2mm'` command. Note that the effects of the `\parindent` and `\parskip` commands are global, which will act on all the paragraphs preceded by them⁵. These can be viewed in paragraphs (2), (3), and (4). Although the `\noindent` command is used before paragraph (3), paragraph (4) is indented by 8 mm following the `\parindent = 8mm` command used before paragraph (2). On the other hand, the `\paragraph{}` command prints its argument as the heading of paragraph (5). Moreover, the `\paragraph{}` command starts paragraph (5) without any indentation and also separates it from paragraph (4) by some predefined vertical blank space. The effect of the `\subparagraph{}` command, shown in paragraph (6), is similar with that of the `\paragraph{}` command, the only difference is that it maintains some indentation at the beginning of the paragraph.

3.6 Creating and Filling Blank Space

Excess blank spaces, created by pressing the `spacebar` or `tab` button of the keyboard, are just ignored in LaTeX, i.e., a sequence of blank spaces is treated as a single one only (similar to converting a sequence of blank lines into a single new line as discussed in §3.5.1). LaTeX provides its own commands for creating a blank space of a specified size, both in horizontal and vertical directions⁶, which are given in

⁵The `\parindent` and `\parskip` commands are generally used in the preamble to make their effects globally in all the paragraphs of a document.

⁶Excess blank spaces cannot be created by pressing the `spacebar` or `tab` button of the keyboard, rather LaTeX provides some commands for creating blank spaces of specified sizes both in horizontal and vertical directions.

Table 3.6 Creating blank spaces

Command	Package	Function	Application	
<code>\quad</code>	—	Predefined horizontal single space	<code>x\quad_{\!y}</code>	$x\ y$
<code>\qquad</code>	—	Predefined horizontal double space	<code>x\qquad_{\!y}</code>	$x\ \ y$
<code>\,</code> or <code>\thinspace</code>	—	Predefined horizontal thin space	<code>x\,_{\!y}</code>	$x\,y$
<code>\:</code> or <code>\medspace</code>	amsmath	Predefined horizontal medium space	<code>x\:_y</code>	$x\,y$
<code>\;</code> or <code>\thickspace</code>	amsmath	Predefined horizontal thick space	<code>x\;_y</code>	$x\,y$
<code>\!</code>	amsmath	Predefined horizontal negative thin space	<code>x\!_y</code>	xy
<code>\!\!</code>	amsmath	Predefined horizontal negative medium space	<code>x\!\!_y</code>	$\!xy$
<code>\!\!\!</code>	amsmath	Predefined horizontal negative thick space	<code>x\!\!\!_y</code>	$\!y$
<code>\bigskip</code>	—	Predefined vertical space	refer Table 3.7	
<code>\vskip</code>	—	User defined vertical space	refer Table 3.7	
<code>\vspace{}</code>	—	User defined vertical space	refer Table 3.7	
<code>\hspace{}</code>	—	User defined horizontal space	refer Table 3.7	
<code>\vfill</code>	—	Vertical space to fill up a page	—	
<code>\hfill</code>	—	Horizontal space to fill up a line	refer Table 3.7	

Table 3.6 (commands of the first part are applicable in both text-mode and math-mode). Note in Table 3.6 that there should not be any blank space on any side of `\,`, `\:`, `\;`, `\!`, `\!\!`, and `\!\!\!` commands; while `\quad` and `\qquad` commands must be followed by a blank space (a command, ended by an alphabet and followed by another alphabet, must be followed by a blank space in order to avoid the formation of a different command which even may not exist). The need of a blank space after a command, ended by an alphabet and followed by another alphabet, can be avoided by writing the following alphabet or word in `{}`, e.g., `'x\quad{y}'` to produce the same output as that by `'x\quad_{\!y}'`. The commands ended by a punctuation can also be applied in this way, e.g., `'x\,{_y}'` in place of `'x\,y'`.

Some applications of the commands given in the second part of Table 3.6 are shown in Table 3.7, where the `\bigskip` command followed by the line break

Table 3.7 Applications of some blank space creating commands

L ^A T _E X input	Output
<code>\begin{center}</code> <code>\LaTeX</code> in 24 Hours \bigskip\ A Practical Guide for Scientific Writing <code>\end{center}</code>	L ^A T _E X in 24 Hours A Practical Guide for Scientific Writing
<code>\begin{center}</code> <code>\LaTeX</code> in 24 Hours <code>\vskip 8mm</code> A Practical Guide for Scientific Writing <code>\end{center}</code>	L ^A T _E X in 24 Hours A Practical Guide for Scientific Writing
<code>\begin{center}</code> <code>\LaTeX</code> in 24 Hours <code>\vspace{8mm}\</code> A Practical Guide for Scientific Writing <code>\end{center}</code>	L ^A T _E X in 24 Hours A Practical Guide for Scientific Writing
Writing language: <code>\hspace{5mm}</code> English.	Writing language: English.
Marks: 100 <code>\hfill</code> Time: 3 Hours.	Marks: 100 Time: 3 Hours.

command `\` (or any other new line command addressed in §3.5.1) creates a predefined vertical positive space between two lines. In contrary, the `\vskip` or `\vspace{}` command creates a user-defined vertical space between two lines or paragraphs. Similarly, the `\hspace{}` command produces a user-defined horizontal space between two words. Note that the argument of the `\vskip` command is written with a blank space, while those of the `\vspace{}` and `\hspace{}` commands are written in `{}`. Moreover, the `\vspace{}` command is to be followed by the line break command `\`, otherwise some texts from the next line may be shifted to the previous line to fill up its blank space, if any.

The unit of the rigid lengths in the arguments of the `\vskip`, `\vspace{}` and `\hspace{}` commands can be any one of `mm` (millimeter), `cm` (centimeter), `in` (inch), `pt` (point), `em` (width of M) and `ex` (width of x). The unit `em` is usually preferred for horizontal lengths, while `ex` for vertical lengths⁷. Apart from these units, a length can also be taken as a fraction of `\textheight` (height of texts on a page), `\textwidth` (width of texts on a page) or `\linewidth` (width of a column), e.g., `0.2\textheight` for a vertical space of 20% of `\textheight` or `0.3\linewidth` for a horizontal space of 30% of `\linewidth`.

L^AT_EX removes blank spaces from the beginning and end of a line as well as a page, for which the `\hspace{}` and `\vspace{}` commands may not have any effect in such places, i.e., at the beginning and end of a line or a page. If so, the commands may be preceded by a `~`. Alternatively, the starred forms of the commands, i.e., the `\hspace*{}` and `\vspace*{}` commands, may be used for producing blank spaces at the beginning or end of a line and a page, respectively. Similarly, instead of the `\hfill`⁸ and `\vfill` commands (abbreviations of the `\hspace{\fill}` and `\vspace{\fill}` commands respectively), the `\hspace*{\fill}` or `\vspace*{\fill}` command is to be used for filling a line (or a page) by creating necessary horizontal (or vertical) space at the beginning or end of the line (or the page).

Note that a positive valued length argument to the `\vskip`, `\vspace{}` or `\hspace{}` command will create a positive blank space, while a negative valued length argument will reduce space by that amount, e.g., `\hspace{-7mm}` will reduce the space between two words by 7 mm (words may get overlapped also, refer foot note 4 on page 12). The `\vskip` or `\vspace{}` command with a negative value may be used (particularly before and after equations, listed items, tables, figures, etc.) to shrink vertical space enabling to accommodate more amount of materials on a page (§5.1.3 on page 39 describes other techniques for the same purpose).

Further, note that the space created by `\hspace{}` may be limited to the available space in a line even for a larger input value in `\hspace{}`, while the space created by `\hspace*{}` with a larger input value (either between two words or at one side of a line) will always continue beyond the page width or even to the next line.

⁷L^AT_EX accepted units for rigid lengths include `mm` (millimeter), `cm` (centimeter), `in` (inch), `pt` (point), `em` (width of M) and `ex` (width of x), where `em` is usually preferred for horizontal lengths and `ex` for vertical lengths.

⁸The `\dotfill` and `\hrulefill` commands are similar to the `\hfill` command, except they fill the gap by dots and horizontal lines respectively.

3.7 Producing Dashes Within Texts

L^AT_EX provides dashes of three different lengths: -, –, and —, which are produced by -, -- and ---, respectively. Out of these dashes, the shortest one is used between inter-related words, the medium one is used to indicate a range, while the longest one is used to show the extension of an expression. Example of each of these three dashes is shown in Table 3.8.

Table 3.8 Dashes of different lengths

L ^A T _E X input	Output
Inter-related	Inter-related
May-August	May–August
Weather condition --- like clear sky	Weather condition — like clear sky

3.8 Preventing Line Break*

Because of the full alignment in a predefined page width, a long statement may need to be split between two words, or even a single word into two parts, continuing the rest in the next line. However, sometime the splitting of a statement between two particular words may need to be prevented, e.g., it is not desirable to split ‘Dilip Datta’ into ‘Dilip’ and ‘Datta’, and then to place them in two lines. Such phrases can be forced to be printed in the same line by using the ~ or \ command between the words without any gap, e.g., ‘Dilip~Datta’ or ‘10\inch’. The \nolinebreak is another command for preventing line breaking at a particular point, e.g., Dilip\nolinebreak__Datta’, where _ means a blank space (the \nolinebreak command is to be followed by _, refer § 1.5.1 on page 5 for detail).

The ~, \, and \nolinebreak commands, however, are always not appropriate, particularly when a word is also to be prevented from splitting. If these commands are set manually between two letters of a word, some undesirable gap may get generated within the word. In that case, the appropriate option is to use the \mbox{} command, which prevents its argument from splitting in two lines⁹. For example, ‘NSGA-II-UCTO’ can be forced to be printed in the same line using ‘\mbox{NSGA-II-UCTO}’. Note that the ~, \, \nolinebreak and \mbox{} commands may sometime print texts beyond the page width. So, it would be a good practice to check the output after the use of such commands.

3.9 Adjusting Blank Space After a Period Mark*

Generally a sentence is ended by a period (full-stop) mark (.) and the next sentence is started with some prespecified gap after that period mark. Sometime a period mark is to be used within a sentence also, e.g., ‘Dr. Datta’, which L^AT_EX will interpret

⁹The \mbox{} command may be used to prevent its argument from splitting in two lines.

as the end of a sentence and hence will print ‘Dr.’ and ‘Datta’ with a big gap between them. Such a big gap can be reduced by commands like `\,` and `\~` without any blank space on any side (reduction of space is minimum under the `\,` command), or `\_` (`_` means a blank space). If a period mark is followed by a right parenthesis or a right quote, the `\` command is to be used after the parenthesis or the quote, otherwise the combination of `\` and parenthesis/quote will be treated as a different command. On the other hand, when a sentence is ended by an uppercase letter, the next sentence is started with a smaller gap. For maintaining a proper gap in this case, the `\@` symbol may be used in between the said uppercase letter and the following period mark. Some applications of these commands are shown in Table 3.9.

Table 3.9 Maintaining proper gap after a period (full-stop) mark

L ^A T _E X input	Output
Dr. Datta	Dr. Datta
Dr. <code>\,</code> Datta	Dr.Datta
Dr. <code>\~</code> Datta	Dr. Datta
Dr. <code>_</code> Datta	Dr. Datta
classes (article, letter, etc.) may be used	classes (article, letter, etc.) may be used
classes (article, letter, etc.) <code>\</code> may be used	classes (article, letter, etc.) may be used
made in USA. United ...	made in USA. United ...
made in USA <code>\@</code> . United ...	made in USA. United ...

3.10 Hyphenating a Word*

If the last word of a line goes beyond the page width, it is generally full-aligned forcibly by breaking the word into two parts with a hyphen to the first part and then putting the second part in the next line. However, in many cases L^AT_EX may continue such a word beyond the page width, or break it in an unpleasant place. Proper word-breaking in that case may be specified manually by inserting the `\-` command in between a word for forcibly hyphenating at that point, e.g., ‘kind`\-`ness’ to print ‘kind-’ at the end of the current line and ‘ness’ at the start-ing of the next line. If L^AT_EX fails to provide proper hyphenation in a paragraph, particularly in the case of a non-dictionary word, it can be put in the `sloppypar` environment also, e.g., `\begin{sloppypar}Maneswarirabha\end{sloppypar}`. However, it would be a good practice to check the output and subsequently to do some manual adjustment, if required.

Formatting Texts II

The basic formatting of a document is discussed in detail in Hour 3. Some advanced formatting, including default as well as user-defined settings, are discussed in this Hour, like foot note, multiple columns, mini page, marginal note, modified sectional unit, etc.

4.1 Increasing Depth of Sectional Units*

As mentioned in §3.1 on page 15, L^AT_EX assigns a three-tier serial numbers to the sectional units, which are `\chapter{}`, `\section{}`, and `\subsection{}` in the document-class **book** or **report**, and `\section{}`, `\subsection{}`, and `\subsubsection{}` in the document-class **article** (the document-class **letter** does not support any sectional unit command). Additionally, the `\part{}` command can be used in the document-class **book**, **report**, and **article** for dividing a document into parts. In the document-class **book** or **report**, the levels of `\part{}`, `\chapter{}`, `\section{}`, and `\subsection{}` commands are -1, 0, 1, and 2 respectively, while those of the `\part{}`, `\section{}`, `\subsection{}`, and `\subsubsection{}` commands in the document-class **article** are 0, 1, 2, and 3 respectively. In order to numbering a few more or less levels, the `\addtocounter{secnumdepth}{}` or `\setcounter{secnumdepth}{}` command may be used in the preamble, e.g., `\addtocounter{secnumdepth}{1}` will increase the numbering one level ahead or `\setcounter{secnumdepth}{5}` will extend the numbering up to the `\subparagraph{}` command. Figure 4.1 shows how the numbering up to the `\subparagraph{}` command

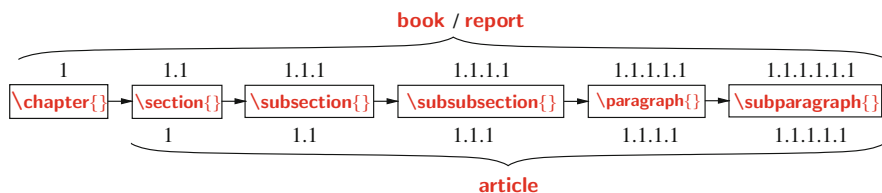


Fig. 4.1 Increasing depth of sectional units

would be when `\setcounter{secnumdepth}{5}` is used. If a particular sectional unit is not to be numbered, commands like `\chapter*`, `\section*` or `\subsection*` may be used. On the other hand, if none of the sectional units is to be numbered, instead of using a starred command every time, simply the `\setcounter{secnumdepth}{}` command may be used with a negative argument, e.g., `\setcounter{secnumdepth}{-6}`.

4.2 Changing Titles and Counters of Sectional Units*

L^AT_EX prints sectional units in certain standard patterns. The serial number (in an Arabic numeral) of a chapter is preceded by the label-word ‘Chapter’, while those of others by none. Some existing commands can be redefined to change these standard patterns. Such an example is shown in Table 4.1. The first `\renewcommand{}`

Table 4.1 Changing titles of sectional units

L^AT_EX input	<pre>\documentclass{book} % \renewcommand{\chaptername}{Unit} \renewcommand{\thechapter}{\Alph{chapter}} \renewcommand{\thesection}{\S~\thechapter.\arabic{section}} \renewcommand{\thesubsection}{\thesecion.\arabic{subsection}} % \begin{document} \chapter{Changed Headings} \section{New form of Section} ... \subsection{New form of Sub-section also} ... \end{document}</pre>
Output	Unit A Changed Headings § A.1 New form of Section ... § A.1.1 New form of Sub-section also ...

command in the preamble redefines the `\chaptername` command to `Unit` (as done in this book to `Hour`), i.e., the label-word of a chapter is changed from `Chapter` to `Unit` (the argument may be left empty if no label-word is required). The second `\renewcommand{}` command redefines the `\thechapter` command to `\Alph{chapter}` to print the serial number of a chapter by an uppercase alphabet¹. On the other hand, the third `\renewcommand{}` command is used to redefine the `\thesection` command

¹The default printing styles of sectional units can be altered by redefining some commands, such as `\chaptername`, `\thechapter`, `\thesection`, `\thesubsection`, and `\thesubsubsection`.

to `\S~\thechapter.\arabic{section}`, where the `\thesection` command controls the numbering of sections. The `\S` command makes the serial number of a section to be preceded by the symbol § with a trailing blank space (due to the following ~ symbol). The serial number is generated through the `\thechapter.\arabic{section}` command, where the `\thechapter` command gives the serial number of the chapter under which the section belongs, followed by a full-stop mark (due to . symbol), and then the `\arabic{section}` command prints the serial number of the section by an Arabic numeral. The last `\renewcommand{}}{}` command redefines the `\thesubsection` command to `\thesection.\arabic{subsection}` for numbering a subsection in an Arabic numeral, preceded by the number of its section (through the `\thesection` command) and a full-stop mark (due to . symbol). Similarly, the default numbering style of other sectional units can also be altered. The other available commands for altering the serial numbers of sectional units include `\alph{}` for numbering in lowercase alphabets, and `\Roman{}` and `\roman{}` for numbering, respectively, in upper and lower Roman numerals.

On the other hand, each type of sectional unit is numbered serially starting from unity. This standard numbering can be changed to start from any intermediate integer through the `\setcounter{asec}{n}` command, where *asec* is the name of the sectional unit like `chapter` or `section`, and *n* is the starting counter of the sectional unit (default value of *n* is 0), which is incremented by 1 each time the sectional command is used². For example, `\setcounter{chapter}{3}` will start the chapter numbering from 4.

4.3 Multiple Columns

The document-classes `article`, `book`, and `report` produce a document in single columns under the default option `onecolumn`. Still they permit to use the `twocolumn` option to `\documentclass[{}]` for producing a document in two columns. If required, some portion of a two-column document can also be produced in a single column by writing that portion in `[]` of the `\twocolumn[]` command (detail is in §19.2.4 on page 187). However, `\twocolumn[]` prints each single-column part on a new page even if sufficient blank space is left on the previous page of the document. Therefore, it is not a good practice to mix up both the options in a single document, except the cases as shown in Table 19.10 on page 188, where the single-column mode is used for printing the abstract of an article in a single column.

²The default starting of numbering of a sectional unit from unity can be altered to start from the intermediate number *n*+1 by inserting `\setcounter{asec}{n}`, where *asec* is the name of the sectional unit like `chapter` or `section`.

4.3.1 Multiple Columns Related Parameters

The visual appearance of a multi-column document may be controlled by the `\columnsep`, `\columnseprule` and `\columnwidth` commands. The `\columnsep` command specifies the gap between two columns, while `\columnseprule` specifies the width of the vertical line separating two columns. On the other hand, `\columnwidth` governs the width of a column and its value is automatically calculated from the values of `\columnsep` and `\textwidth`.

The default value of `\columnseprule` is set as zero to make the vertical line invisible, while the value of `\columnsep` varies with the used document-class and font size (detail is in §5.1.1 on page 37). Their default effects can be altered by assigning them new values in the same way as shown in Table 5.3 on page 40 for page formatting commands. Table 4.2 shows an example of a two-column document, where a vertical line is generated between the columns by assigning a value of 1 mm to `\columnseprule`.

Table 4.2 Two columns separated by a vertical line through the `\columnseprule` command

L ^A T _E X input	Output
<pre>\documentclass[twocolumn]{article} \columnseprule = 1mm % \begin{document} The standard document-classes permit to print a document either in a ... \end{document}</pre>	The standard document-classes permit to print a document either in a single-column or in a two-column mode. This is an example of a two-column mode where the columns are separated by a vertical line.

4.3.2 A Flexible Approach to Generate Multiple Columns

The standard L^AT_EX can produce a document in a maximum of two columns through the `twocolumn` option to the `\documentclass[{}]{}` command. Moreover, although the `\twocolumn[ ]` command can produce a portion of a two-column document in a single column, each single column is printed on a separate page. On the other hand, the columns on the last page are not balanced. To overcome such problems, the `multicols` environment, defined in the `multicol` package, can be used, which has the capability of producing any number of columns (up to ten) even on a single-column page³. Moreover, it can be applied inside other environments, like `minipage` (detail is in §4.4 on the following page). Table 4.3 on the facing page shows an application

³The `multicols` environment, defined in the `multicol` package, can be used for producing any number of columns (up to ten) even on a single-column page.

Table 4.3 Multiple columns generated through the **multicols** environment

L ^A T _E X input	Output
<pre>\documentclass[11pt,a4paper]{article} \usepackage{multicol} \columnseprule = 0.5mm % \begin{document} This is an example where a ... % \begin{multicols}{3} This is a three-column paragraph. ... \end{multicols} % In this example, all the three ... \end{document}</pre>	This is an example where a paragraph is written in three columns. The paragraphs above and below it are written in single columns. This is a three-column paragraph. The gap between two columns are filled by a vertical line of 0.5 mm width. In this example, all the three paragraphs are probreakduced on the same page.

of the **multicols** environment, where three columns on a single-column document are generated through the mandatory argument of the environment. Moreover, two adjacent columns are separated by a vertical line of 0.5mm width through **\columnseprule**. Because of the default full alignment of texts, excess blank space can be seen in the columns in Table 4.3. In order to avoid such excess blank space, texts may be made left aligned using **\raggedright**, which is to be inserted just after the **\begin{multicols}{3}** command.

In Table 4.3, the multi-column portion was shown preceded by a single-column paragraph. Instead of inserting in that way, it can also be inserted as an optional argument to the **multicols** environment as **\begin{multicols}{3}[apref]**, where **apref** is the piece of texts to precede the multi-column portion.

The columns under the **multicols** environment can be customized through **\columnsep** and **\multicolsep**. The **\columnsep** command specifies the gap between two columns, while **\multicolsep** controls the vertical gap before and after the environment. The default values of **\columnsep** and **\multicolsep** in this environment are around 3.5 mm and 4.5 mm, respectively. The commands can be assigned other values in the same way as shown in Table 5.3 on page 40 for page formatting commands. On the other hand, the width of a column is calculated from the number of columns to be generated and the current value of **\linewidth**. The **\linewidth** command specifies the width of the current lines of texts, whose value is usually the same with that of **\textwidth** (detail is in §5.1.2 on page 38), but may vary inside some environments like **quotation** (refer §3.4 on page 18 for detail).

The **table** and **figure** environments (details are in Hours 7 and 9) may not be supported properly in the **multicols** environment. In that case, their starred forms, i.e., **table*** and **figure*** environments, may be used for drawing tables or figures spanning all the columns.

4.4 Mini Pages

Mini pages mean dividing a portion of a page into a number of width-wise parts, usually for presenting related materials side-by-side, e.g., two tables for comparison purpose, a picture and its description, or L^AT_EX input and its output. Mini pages are

generated through the `minipage` environment with a pair of optional and mandatory arguments, i.e., `\begin{minipage}[avert]{ahorz}`, where `avert` is the vertical alignment of the mini page and `ahorz` is its horizontal width⁴. The allowed vertical alignment options are center (default), top, and bottom, which are denoted by `c`, `t`, and `b`, respectively. In the case of the horizontal width, the sum of widths of all the side-by-side mini pages should not exceed the line width of the main page, otherwise the excess mini pages will be placed in the following lines. Hence, instead of specifying any fixed numeric value (say, 5cm), the width of a mini page may be specified as a fraction of the `\linewidth` command, say `'0.4\linewidth'` to create a mini page of width equal to 40 % of the line width of the main page.

Similar to the `minipage` environment, there exists `boxedminipage` environment also, which is defined in the `boxedminipage` package. The basic difference between the environments is that each mini page created by the `boxedminipage` environment is placed in a box. Applications of the environments are shown in Table 4.4, where

Table 4.4 Dividing a page width-wise using the `minipage` and `boxedminipage` environments

LaTeX input	Output
<pre>\documentclass[11pt,a4paper]{article} \usepackage{boxedminipage} % \begin{document} % \begin{minipage}[t]{5.0cm} This minipage is ... width is 5.0\cm. \end{minipage} % \begin{boxedminipage}[t]{0.4\linewidth} This boxed-minipage is also ... \end{boxedminipage} % \end{document}</pre>	This minipage is top-aligned and its horizontal width is 5.0cm. This boxed-minipage is also top-aligned, but its horizontal width is equal to 40% of the page line-width.

both the environments are top-aligned with `minipage` of 5.0cm width and that of `boxedminipage` is equal to 40 % of the line width of the main page.

4.5 Foot Notes

LaTeX provides the `\footnote{}` command for printing its argument as a foot note. The command is to be inserted just after the word or phrase (usually without any gap) against which a foot note is to be generated. In the output, such words or phrases are superscribed in Arabic numerals with the counter of the `\footnote{}` command. The corresponding foot notes are printed serially at the bottom of the pages and

⁴The `minipage` environment can be used for dividing a portion of a page into a number of width-wise parts, usually for presenting related materials side-by-side, e.g., two tables for comparison purpose, or a picture and its description.

they are marked by superscribing the respective serial numbers on the left side. As shown in Table 4.5, the foot notes on a page are separated from its main materials

Table 4.5 Foot notes generated through the `\footnote{}` command

LaTeX input	Output
Both Rubi and Lila\footnote{They are sisters.} study in class I, while Ravi and Joy\footnote{They are friends.\label{fn:friends}} study in class II.	Both Rubi and Lila ¹ study in class I, while Ravi and Joy ² study in class II. ¹ They are sisters. ² They are friends.

by a small horizontal line. A foot note may contain anything, such as paragraphs, lists, mathematical expressions, tabular materials, etc. Generally a word/phrase and the foot note generated against it appear on the same page. However, a foot note may be continued on the following pages also if it is long enough. On the other hand, the foot notes of a multi-column document are printed in full width at the bottom of a page, irrespective of the columns where from they are generated.

A foot note can be labeled and referred like other numbered items. As shown in Table 4.5, a foot note is to be labeled using `\label{}` inside `{}` of the `\footnote{}` command, and it can be referred as usual by the `\ref{}` command⁵.

4.5.1 Foot Notes in Mini Pages*

The markings and positions of foot notes inside a `minipage` environment are quite different. The foot notes of each mini page are marked by lowercase alphabets (a, b, c, etc.) and placed them at the bottom of the mini page. An example of foot notes in mini pages, along with a foot note in the main page also, is shown in Table 4.6 on the following page. The line break command `\[2mm]` is used before the first and after the last `minipage` environments for producing the mini pages in a separate paragraph with a gap of 2 mm before and after the paragraph. Moreover, the `\hfill` command is used after the first `minipage` environment for placing the mini pages left and right aligned with the remaining space of the line width of the main page in between the mini pages. As seen in Table 4.6, the foot note in the main page and those inside the mini pages are counted separately (by Arabic numerals in the main page, while by lowercase alphabets in the mini pages).

⁵Like other numbered items, a foot note generated through `\footnote{}` can also be labeled and referred using `\label{}` and `\ref{}`, respectively.

Table 4.6 Foot notes in mini pages

LaTeX input	Output
<pre>The following is an example of ... mini page\footnote{Foot note style in mini pages is different.}\[2mm] % \begin{minipage}[t]{0.45\linewidth} Both Rubi and Lila\footnote{They are sisters.} study ... Joy\footnote{They are friends.} study in class II. \end{minipage}\hfill % \begin{minipage}[t]{0.45\linewidth} Though the milk of a cow\footnote{Domestic animal.} is ... milk of a tiger\footnote{Wild animal.}. \end{minipage}\[2mm] % Foot notes in a mini page are marked by lowercase alphabets and placed ...</pre>	The following is an example of foot notes inside a mini page¹. Both Rubi and Lila^a study in class I, while Ravi and Joy^b study in class II. ^aThey are sisters. ^bThey are friends. Though the milk of a cow^a is used in various food products, nobody bothers about the milk of a tiger^b. ^aDomestic animal. ^bWild animal. Foot notes in a mini page are marked by lowercase alphabets and placed at the bottom of the mini page. ¹Foot note style in mini pages is different.

4.5.2 Altering the Pattern of Foot Notes*

By default, foot notes are numbered in Arabic numerals. Other markings can be obtained by redefining the `\thefootnote` command in the preamble. For example, the `\renewcommand{\thefootnote}{\fnsymbol{footnote}}` command may be used for marking the foot notes by symbols like *, †, ‡, §, etc.

Although the markings of foot notes inside mini pages can also be altered by changing their counter `mpfootnote` by redefining the `\thempfootnote` command, still the markings are started afresh in each mini page. Such markings may be confusing in many cases, particularly if a foot note is to be referred. LaTeX provides the `\footnotemark` and `\footnotetext{}` commands through which the foot note of a mini page can also be made to work like foot notes in main pages⁶. That is, the commands alter the counter of the foot notes of a mini page from `mpfootnote` to `footnote`, which is the counter of foot notes of main pages. The application of the `\footnotemark` and `\footnotetext{}` commands is shown in Table 4.7 on the next page. Like the `\footnote{}` command in main pages, the `\footnotemark` command in a `minipage` environment is used just after the word or phrase against which a foot note is to be generated. Then the contents of the foot note are written as the argument of the `\footnotetext{}` command immediate after the `minipage` environment. A foot note of a mini page, generated through the `\footnotemark` and `\footnotetext{}` commands, can be labeled and referred as an ordinary foot note. In that case, as shown in the second `minipage` environment in Table 4.7, the foot note is to be labeled inside `{}` of the `\footnotetext{}` command. However, this process for generating foot notes in mini pages has the drawback that only one foot note can be generated against a `minipage` environment.

⁶LaTeX provides `\footnotemark` and `\footnotetext{}` for generating foot notes in mini pages, similar to those in main pages generated through `\footnote{}`.

Table 4.7 Foot notes of a mini page to work like those in main pages

LaTeX input	Output
<pre>The following is an example of ... mini page\footnote{Foot note style in mini pages is different.}\[2mm] % \begin{minipage}[t]{0.45\linewidth} Both Rubi and Lila\footnotemark\study in class I. \end{minipage} \footnotetext{They are sisters.}\hfill % \begin{minipage}[t]{0.45\linewidth} The milk of a cow\footnotemark\is ... \end{minipage} \footnotetext{Domestic animal.\label{fn:cow}}\[2mm] % Foot notes in a mini page are marked ...</pre>	The following is an example of foot notes inside a mini page¹. Both Rubi and Lila² The milk of a cow³ is study in class I. used in various food products. Foot notes in a mini page are marked by lowercase alphabets and placed at the bottom of the mini page. ¹Foot note style in mini pages is different. ²They are sisters. ³Domestic animal.

The inclusion of the `fnpara` package prints all the foot notes of a page in a single paragraph, instead of stacking them one below another. To put all the foot notes in the right column in a two-column document, the `fnright` package may be loaded.

Similar to the `\footnote{}` command, the `\endnote{}` command under the `endnotes` package may be used for printing notes at the end of a chapter or document under the heading ‘Notes’. Foot notes can also be printed as end notes by redefining the `\footnote` command as `\renewcommand{\footnote}{\endnote}`.

4.6 Marginal Notes*

LaTeX provides the `\marginpar{}` command for printing an important note in the margin of a document. An application of the command can be seen just outside of this line, which is produced by inserting ‘`\marginpar{\em Marginal note.}`’ after the word `line` in the current sentence of this book. A marginal note is started against the line of a page in which the `\marginpar{}` command appears.

The `\marginpar{}` command usually prints its argument as a marginal note in the right margin of a page. In the case of a `twoside` document, such as `book`, a marginal note is printed in the right margin of an odd numbered page and in the left margin of an even numbered page. In a `twocolumn` document, a marginal note is printed in the nearest margin. Although LaTeX decides the correct margin on its own without bothering a user, a problem may arise if an arrow is to be used in the margin to point some texts. This is because, as shown in the margin of this line, a left-pointing arrow is required in the right margin and a right-pointing one is required in the left margin. However, it is not known beforehand in which margin the arrow will appear, for which it would be confusing whether a right-pointing or a left-pointing arrow is

*Marginal
note.*



⇒

to be used. L^AT_EX avoids this confusion also by allowing the `\marginpar{}` command to take an optional argument, which is processed if the marginal note is to appear in the left margin, otherwise the mandatory argument is processed to print the marginal note in the right margin. For example, the arrow in the margin of this line is obtained through the `\marginpar{${\Longrightarrow}${\Longleftarrow}}` command.

When the `\marginpar{}` command is used for printing a marginal note, the page size of a document is to be adjusted to accommodate the marginal notes. The commands required in this regard include `\marginparwidth`, `\marginparsep`, and `\marginparpush`. These commands signify, respectively, the width of a marginal note, the horizontal gap between the main texts of a page and a marginal note, and the vertical gap between two successive marginal notes. As addressed in §5.1.2 and shown in Table 5.3 on page 40, all of these three commands are to be put in the preamble, with appropriate values, along with other page setting commands. As an example, in this book, the `\marginparwidth`, `\marginparsep`, and `\marginparpush` commands are assigned the values of 12 mm, 3 mm, and 15 mm, respectively.

Page Layout and Style

L^AT_EX has default page layouts and styles which can be used without any difficulty. These default settings can also be customized, if required, which are discussed in this Hour.

5.1 Page Layout

The type of the physical paper for producing a document can be specified as an option to the `\documentclass[]{}` command, e.g., `\documentclass[a4paper]{article}` for printing an article on A4-size paper (**a4paper** is the type of paper). The types and sizes of the standard papers accepted by the `\documentclass[]{}` command are listed in Table 5.1. The length of a line and the number of lines per page for printing

Table 5.1 Types and sizes of standard papers accepted by the `\documentclass[]{}` command

Type of paper	Size	Type of paper	Size
a4paper	210 mm×297 mm	letterpaper (default)	216 mm×279 mm
a5paper	148 mm×210 mm	legalpaper	216 mm×356 mm
b5paper	176 mm×250 mm	executivepaper	184 mm×267 mm

texts are fixed according to the chosen paper-size. Besides the size of a paper, its orientation can also be specified by **portrait** (default) or **landscape** as another option to `\documentclass[]{}` . In the **portrait** orientation, the longer dimension of a page goes in vertical and shorter dimension in horizontal, while the **landscape** orientation is just opposite to the **portrait** orientation.

5.1.1 Standard Page Layout

There are many L^AT_EX commands controlling various dimensional parameters of the page layout for a document. The commands controlling some important parameters are listed in Table 5.2 on the next page, as well as shown diagrammatically

Table 5.2 Commands controlling some important parameters of a page layout

Layout	Command	Function
Main texts	<code>\textheight</code>	Height of main texts without header and footer.
	<code>\textwidth</code>	Width of main texts without marginal notes.
	<code>\oddsidemargin</code>	Blank space on the left margin of odd-numbered pages, if both-side printing is opted (both-side printing can be opted using the <code>\twoside</code> option to the <code>\documentclass[]{ }</code> command).
	<code>\evensidemargin</code>	Blank space on the left margin of even-numbered pages (active only when both-side printing is opted).
Columns	<code>\columnsep</code>	Gap between two columns in multi-column mode.
	<code>\columnseprule</code>	Width of the vertical line separating two columns in multi-column mode (default value is zero so as to make the line invisible).
	<code>\columnwidth</code>	Width of a column in multi-column mode (calculated using the values of <code>\textwidth</code> and <code>\columnsep</code>).
	<code>\linewidth</code>	Width of the lines of texts (usually equal to <code>\columnwidth</code> , but may vary in some environments, like the <code>\quotation</code> environment).
Header	<code>\headheight</code>	Height of the header.
	<code>\headsep</code>	Vertical gap between the header and the first line of the main texts.
	<code>\topmargin</code>	Extra vertical space above the header.
Footer	<code>\footskip</code>	Vertical gap between the last line of the main texts and the footer.
Marginal notes	<code>\marginparwidth</code>	Width of marginal notes.
	<code>\marginparsep</code>	Horizontal gap between the main texts and marginal notes.
	<code>\marginparpush</code>	Vertical space between two successive marginal notes.

in Fig. 5.1 on page 39. The `\hoffset` and `\voffset` commands in Fig. 5.1 represent, respectively, the horizontal and vertical coordinates of the reference point, whose default values are usually set to zero. All the standard papers listed in Table 5.1 have some fixed values for the commands controlling the dimensional parameters of a page layout. Even for the same paper, the value of a command may vary with the size of fonts as well as with the type of printing (single-side or both-side). However, the values of these commands are independent of the three standard document-classes of `article`, `book`, and `report`. For a particular setting, the page layout similar to the one shown in Fig. 5.1, along with the values of some parameter controlling commands, can be obtained through the `\layout` command defined in the `layout` package. Such values for the document-class `article` in 12pt fonts on `oneside` printing `a4paper` paper are also shown in Fig. 5.1 (obtained just by executing the `\layout` command), where the values of the commands are given in pt (1pt $\approx$ 0.3515mm). The commands `\paperheight` and `\paperwidth` represent, respectively, the height and width of the physical paper.

5.1.2 Formatting Page Layout*

Instead of using a standard page layout, a user can create an own layout by assigning suitable values to the layout controlling commands. Such a command can be assigned a new value either through an explicit mathematical expression or through the `\setlength{acomm}{aval}` command, where `acomm` is a command and `aval` is its new value. If a command is not assigned a new value, its default value for the

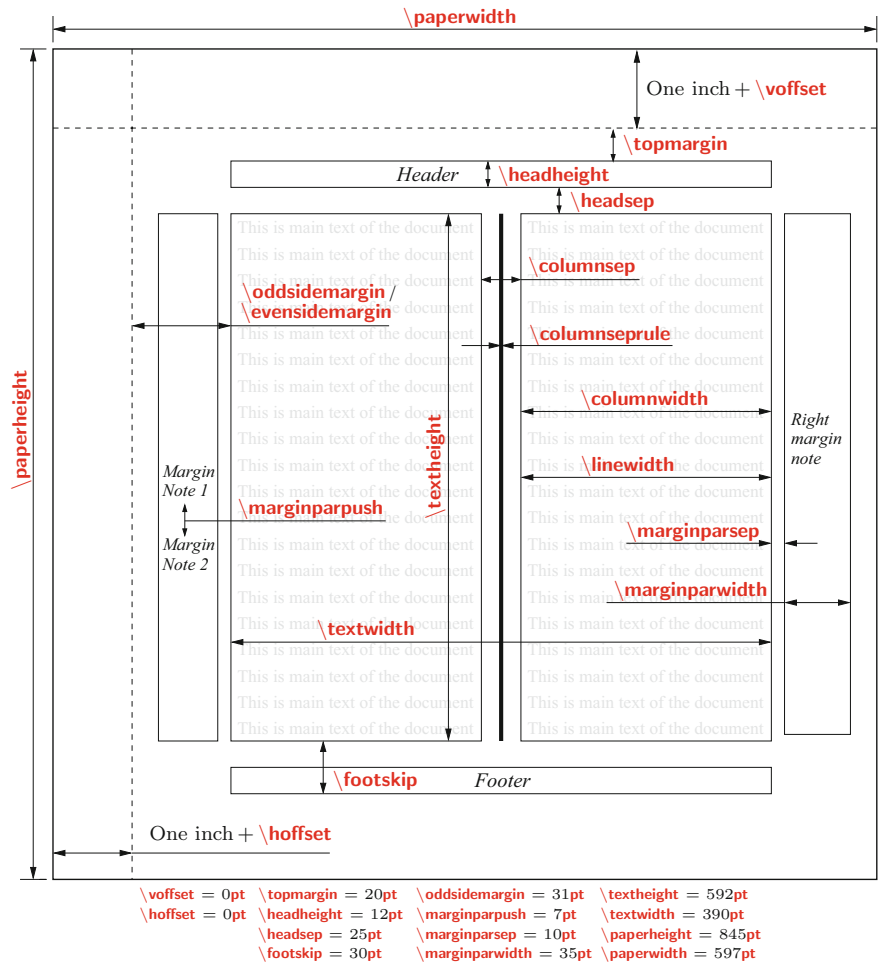


Fig. 5.1 Commands controlling some important parameters of a page layout

chosen paper is used. The values used during drafting this book (the paper is **a4paper**) are shown in Table 5.3 on the next page through both mathematical expression and the `\setlength{...}` command (any one of these forms is to be inserted in the preamble of the input file of a document).

5.1.3 Increasing the Height of a Page*

The commands discussed in §5.1.1 and §5.1.2 format a page layout globally, i.e., their values remain the same on all pages. Sometime, however, the vertical height of a particular page may need to be enlarged locally, specially to accommodate a table

Table 5.3 A manually defined page layout (only one form is to be used in the preamble)

Mathematical expression	Use of the <code>\setlength{}</code> command
<code>\topmargin</code> = 0mm	<code>\setlength{\topmargin}{0mm}</code>
<code>\oddsidemargin</code> = 15mm	<code>\setlength{\oddsidemargin}{15mm}</code>
<code>\evensidemargin</code> = 0mm	<code>\setlength{\evensidemargin}{0mm}</code>
<code>\textheight</code> = 210mm	<code>\setlength{\textheight}{210mm}</code>
<code>\textwidth</code> = 150mm	<code>\setlength{\textwidth}{150mm}</code>
<code>\marginparwidth</code> = 15mm	<code>\setlength{\marginparwidth}{15mm}</code>
<code>\marginparsep</code> = 3mm	<code>\setlength{\marginparsep}{3mm}</code>
<code>\marginparpush</code> = 7mm	<code>\setlength{\marginparpush}{7mm}</code>
<code>\parindent</code> = 5mm	<code>\setlength{\parindent}{5mm}</code>
<code>\footskip</code> = 10mm	<code>\setlength{\footskip}{10mm}</code>

or a figure, or even small piece of texts of a section going to a new page. This can be done through the `\enlargethispage{asize}` or `\enlargethispage*{asize}` command, where `\enlargethispage{}` increases the vertical height of a page by an amount of `asize`, while `\enlargethispage*{}` tries to shrink inter-line spacing to free that amount of space on the page. The commands may be inserted in any place in the input file containing the materials to be printed on the particular page.

5.2 Page Style

Once a page layout is set, the next step is the design of the pages of a document, which mainly includes running header and footer, page numbering, and the front page of a chapter. L^AT_EX provides various standard macros as well as customizing facilities for designing a document. The header and footer, and page numbering are discussed in the following two sections, while the front page of a chapter is covered in §4.2 on page 28.

5.3 Running Header and Footer

The running header and footer on the pages of a document are controlled by the `\pagestyle{}` and `\thispagestyle{}` commands, whose arguments specify a page style. Some commonly used page styles are listed in Table 5.4 on the following page. The `\pagestyle{}` command implements the chosen page style on the current page as well as on the succeeding pages, while `\thispagestyle{}` works locally on the current page only. Usually `\pagestyle{}` is used in the preamble for its global effect, and `\thispagestyle{}` is used inside the `document` environment to suppress the global effect of `\pagestyle{}` on the current page, e.g., `\thispagestyle{empty}` may be used in the title page of a book or a report.

By default, `\thispagestyle{plain}` is issued by the document-classes of `article`, `book`, and `report` to the `\maketitle` command and the first page of major sectioning commands,

Table 5.4 Page styles to control running header and footer in a document

Page style	Package	Function	Application
empty	—	Both the head and foot are empty, and there is no page number (however, the pages are counted).	—
plain	—	The head is empty, but the foot contains the page number at the center of the foot (default in the document-classes article and report).	—
headings	—	The foot is empty, but the head contains the page number as well as title information as determined by a document-class, like chapter and section headings (does not apply to the first page of a chapter).	Refer §5.3.1
myheadings	—	Same with headings except that the page titles in the head are not automatic, but a user has to supply through some commands.	Refer §5.3.2
fancy	fancyheadings	Fully user-defined headers and footers.	Refer §5.3.3
fancy	fancyhdr	Fully user-defined headers and footers.	Refer §5.3.4

like `\part{}` or `\chapter{}`. To suppress the effect of `\thispagestyle{plain}` on these pages, `\thispagestyle{}` with an appropriate page style may be used just after the `\maketitle`, `\part{}` or `\chapter{}` command, e.g., `\thispagestyle{empty}` may be used to suppress the default page numbering on these pages.

5.3.1 Header with the **headings** Style

In the page style **headings**, the running footer is empty, and the header contains the page number and the title of a sectional unit of that page. The position of the page number depends on the type of printing, and the sectional unit whose title appears in the heading depends on the chosen document-class. Different types of headings under the page style **headings** are outlined in Table 5.5. In the case of multiple sectional units on a page, the title of the last unit appears on the header. In the heading, the title of a `\chapter{}` is also preceded by the label-word `CHAPTER` and its serial number, while the title of a `\section{}` or a `\subsection{}` is preceded by its serial number only.

Table 5.5 Styles of headers under the **headings** page style

Option	Page	Document-class		
		article	book	report
oneside (One-side printing)	Odd	Title of <code>\section{}</code> on left and page number on right	Title of <code>\section{}</code> on left, if any, and page number on right	Title of <code>\chapter{}</code> on left and page number on right
	Even	Same with an odd page	Title of <code>\chapter{}</code> on right and page number on left	Same with an odd page
twoside (Two-side printing)	Odd	Title of <code>\subsection{}</code> on left, if any, and page number on right	Same with an odd page of one-side printing	Title of <code>\section{}</code> on left, if any, and page number on right
	Even	Title of <code>\section{}</code> on right and page number on left	Same with an even page of one-side printing	Title of <code>\chapter{}</code> on right and page number on left

5.3.2 Header with the *myheadings* Style

The running header in the page style *myheadings* is the same with that in the page style *headings*. The only difference is that the style of the header under *headings* is predefined, while it is user-defined under *myheadings*. The information about a sectional unit is passed to the running header through a specified marker command, e.g., the marker commands associated with `\chapter{}`, `\section{}`, and `\subsection{}` are `\chaptermark`, `\sectionmark`, and `\subsectionmark`, respectively. Further, the `\markboth{ }{ }` and `\markright{ }` commands are associated with a marker command. The `\markboth{aeven}{aodd}` command is active under the *twoside* printing option with *aeven* as the contents of the header of even (left hand) pages and *aodd* as that of odd (right hand) pages. In `\markboth{ }{ }`, page number is printed on left side of even pages and on right side of odd pages. On the other hand, `\markright{acont}` is appropriate for *oneside* printing option, where *acont* is the contents of the header of all pages.

Generally the information of two different sectional units (e.g., `\chapter{}` and `\section{}`, or `\section{}` and `\subsection{}`) is put on odd and even pages. However, the information of both the sectional units cannot be passed through `\markboth{ }{ }` associated with the marker command of a single sectional unit. Therefore, `\markboth{aeven}{ }` and `\markright{aodd}` are used in combination, where *aeven* is the information for left hand pages and *aodd* is that for right hand pages (the second argument of `\markboth{ }{ }` is not processed). In this combination, *aeven* and *aodd* are internally stored as `\leftmark` and `\rightmark`, respectively.

Once `\markboth{ }{ }` and `\markright{ }` are finalized, they are associated with the marker commands of two different sectional units, where the marker commands are redefined through the `\renewcommand{ }{ }` command to implement the changes made. Two examples of redefining the marker commands `\chaptermark`, `\sectionmark`, and `\subsectionmark` are shown in Table 5.6 (each `\renewcommand{ }{ }`

Table 5.6 Redefining running header generating marker commands

Example	Redefinition of marker commands	Document-class
1	<code>\renewcommand{\chaptermark}[1]% {\markboth{\chaptername~\thechapter. #1}{ }}</code> <code>\renewcommand{\sectionmark}[1]% {\markright{\thesection. #1}}</code>	book / report with twoside printing option
2	<code>\renewcommand{\sectionmark}[1]% {\markboth{\thesection. #1}{ }}</code> <code>\renewcommand{\subsectionmark}[1]% {\markright{\thesubsection. #1}}</code>	article

is split into two lines as `\renewcommand{ }{ }%` and `{ }` due to space limit). The `\chaptername` command generates the label-word *Chapter*, while `\thechapter`, `\thesection`, and `\thesubsection` generate the serial numbers of the current chapter, section and subsection, respectively. The syntax `[1]` means that there is one argument and the syntax `#1` is the location where the argument would be printed (in the current

examples, it might be the title of a sectional unit). The first `\renewcommand{ }{ }` command in example 1 would generate a running header on left hand pages something like ‘Chapter 1. Headers and Footers’, while the second `\renewcommand{ }{ }` command of this example would generate a header on right hand pages something like ‘1.2. Generating Headers’. The redefinitions given in Table 5.6 are default headers under the page style **headings** discussed in §5.3.1. To get the same headers under the page style **myheadings**, these redefinitions are to be put in the preamble of the input file of a document. Since the headers under the page style **myheadings** are user-defined, it is not necessary to use the exact redefinitions given in Table 5.6, but these can be customized as one wishes. For example, the `\chaptername` command in example 1 may be omitted if the label-word `Chapter` is not to be printed in the header, or the headers can be produced in boldface fonts by writing the arguments of `\markboth{ }{ }` and `\markright{ }` through `\textbf{ }`, like `\markboth{\textbf{thechapter. #1}}{ }` and `\markright{\textbf{thesection. #1}}{ }`.

5.3.3 Header and Footer with the **fancy** Style Under the **fancyheadings** Package*

The page style **fancy**, defined in the **fancyheadings** package, allows very elegant customization of the running header and footer of a document. The package provides three types of headers as well as footers, through which a header/footer can be made page-wise left, center or right aligned, or even multiple pieces of headers and footers can be used. The commands for such headers and footers are shown in Table 5.7, where `podd` and `peven` are the contents of the headers/footers on odd and

Table 5.7 Commands for headers and footers under the **fancyheadings** package

Commands for headers	Commands for footers	Alignment
<code>\lhead[peven]{podd}</code>	<code>\lfoot[peven]{podd}</code>	Left
<code>\chead[peven]{podd}</code>	<code>\cfoot[peven]{podd}</code>	Center
<code>\rhead[peven]{podd}</code>	<code>\rfoot[peven]{podd}</code>	Right

even pages, respectively (optional `peven` becomes active only if **twoside** printing is opted). The line break command `\` can also be used to put a header/footer in multiple lines. However, in that case the values of `\headheight` and `\footskip` may need to be increased (refer Fig. 5.1 and Table 5.3).

In the **fancyheadings** package, provision is also there for drawing horizontal rules (lines) below the header and above the footer, whose thicknesses are controlled through the `\headrulewidth` and `\footrulewidth` commands, respectively. The default value of `\headrulewidth` is 0.4 pt, while that of `\footrulewidth` is 0 pt (making the rule above the footer invisible). The thickness of a rule can be altered through `\setlength{ }{ }` (refer Table 5.3 for detail). Moreover, the headers, footers and their rules can also be extended to cover the marginal notes, by increasing the value

of their width controlling command `\headwidth` from `\textwidth` (default value) to `\textwidth+\marginparsep+\marginparwidth`.

The first three pages of a book chapter (the front page, an even page, and an odd page), designed through the page style `fancy` under the `fancyheadings` package, are shown in Table 5.8, where the line numbers in the L^AT_EX input file are shown for explanation purpose only. Two-side printing is opted in line 1 through the `twoside` option to `\documentclass[]{}.` The `fancyheadings` package is loaded in line 2 and the page style `fancy` is opted in line 4. The marker commands `\chaptermark` and `\sectionmark` are redefined in lines 5 and 6 through `\markboth{ }{ }` and `\markright{ }`, respectively. The first argument of `\markboth{ }` and the only argument of `\markright{ }`, which in the

Table 5.8 Header and footer with the `fancy` page style under the `fancyheadings` package

L ^A T _E X input	<pre>1 \documentclass[12pt,a4paper,twoside]{book} 2 \usepackage{fancyheadings} 3 % 4 \pagestyle{fancy} 5 \renewcommand{\chaptermark}[1]{\markboth{\thechapter. #1}} 6 \renewcommand{\sectionmark}[1]{\markright{\thesection. #1}} 7 \head[\textbf{\thechapter}]{\textbf{\rightmark}} 8 \rhead[\textbf{\leftmark}]{\textbf{\thechapter}} 9 \foot[\textbf{Engineering Mechanics}]{ } 10 \rfoot[\textbf{Dilip Datta}] 11 \cfoot[] 12 \renewcommand{\headrulewidth}{0.15mm} 13 \renewcommand{\footrulewidth}{0.15mm} 14 \addtolength{\headwidth}{\marginparsep} 15 \addtolength{\headwidth}{\marginparwidth} 16 % 17 \begin{document} 18 ... 19 \chapter{Distributed Force System} 20 \thispagestyle{empty} 21 For simplifying an analysis, the force exerted by a body... 22 ... 23 \section{Centre of Gravity} 24 Since the weight of a body is a system of concurrent forces... 25 ... 26 \end{document}</pre>		
	Output	46 4. Distributed Force System 4.1 Centre of Gravity Since the weight of a body is a system of concurrent forces with the centre of the earth as the point of concurrency, the weight of the body can be represented by the resultant of the system of forces. The point on the body through which this resultant gravitational force passes is known as the centre of gravity of the body. The centre of gravity of a body can be obtained through a very simple experiment. Tie the body at a point by a string and hang it from a support. Then the body will be in equilibrium under the action of two equal, opposite and collinear forces: the weight of the body and the tension in the string. Mark the line of action of the forces by drawing a lighted hole of a small diameter. In this way, get two or more holes by tying the body at different points. Interestingly, all the holes would be found passing through a single point, which is the centre of gravity of the body. Therefore, the centre of gravity of a body can be defined as the point through which the resultant of the elemental gravitational forces of the body would pass regardless of the orientation of the body in the space. Since the distance between any two elements of a body is usually very very negligible in comparison with the distance from the body to the centre of the earth, Centre of Gravity Engineering Mechanics	47 4.1. Centre of Gravity It may be assumed that the elemental gravitational forces of the body are parallel to each other. Under this assumption, the centre of gravity of a body can be obtained mathematically by applying the principle of moment to a system of parallel forces, which would result the following coordinate for the centre of gravity of the body: $\left(\bar{x} = \frac{\int x \, dW}{\int dW}, \bar{y} = \frac{\int y \, dW}{\int dW}, \bar{z} = \frac{\int z \, dW}{\int dW} \right) \quad (4.1)$ Further, since any internal distance of a body is usually negligible in comparison with the distance from the body to the centre of the earth, it may be assumed that the gravitational field on the body is uniform, giving the same value of the gravitational acceleration at every point of the body (i.e. $g = g_0 = g$ g/m/s²). Therefore, the coordinate of the centre of gravity, given by Eq (4.1), can be simplified as below: $\left(\bar{x} = \frac{\int x \, dW}{\int dW}, \bar{y} = \frac{\int y \, dW}{\int dW}, \bar{z} = \frac{\int z \, dW}{\int dW} \right) \quad (4.2)$ Since the coordinates of Eq (4.2) contain no reference to the gravitational field g, but only to the mass m of the body, the centre of gravity can be termed as the centre of mass under the assumption of a uniform and parallel gravitational field. Centre of Mass Dilip Datta

present case are ‘\thechapter. #1’ and ‘\thesection. #1’, are internally stored in \leftmark and \rightmark, respectively (the second argument of \markboth{} is not processed as mentioned in §5.3.2 on page 42). Various headers and footers, as stated in Table 5.7, are defined in lines 7–11. The arguments of \head[]{} in line 7 print \thepage (page number) and \rightmark (information about \section{}) as the left aligned headers on even and odd pages, respectively. Similarly, the arguments of \rhead[]{} in line 8 print \leftmark (information about \chapter{}) and \thepage (page number) as the right aligned headers on even and odd pages, respectively. Moreover, each of \thepage, \rightmark, and \leftmark is inserted as the argument of \textbf{} for printing the corresponding contents in boldface fonts. On the other hand, values to the first argument of \foot[]{} in line 9 and the second argument of \rfoot[]{} in line 10 are supplied for printing their contents as the left aligned footer on even pages and right aligned footer on odd pages, respectively. No value is assigned to the arguments of \cfoot[]{} in line 11 in order to keep the center footer blank, otherwise the page number will be printed here also. The \renewcommand{}{} command in lines 12 and 13 assigns the value of 0.15mm to each of \headrulewidth and \footrulewidth for obtaining rules, each of thickness 0.15 mm, below the header and above the footer, respectively. Moreover, \addtolength{}{} is used in lines 14 and 15 for adding the values of \marginparsep and \marginparwidth to the default value of \headwidth, which is done for increasing the horizontal widths of the header, footer and rules to cover the marginal notes also. Finally, \thispagestyle{empty} is inserted just after each \chapter{} (shown in line 20) to avoid numbering of the first page of a chapter, otherwise \thispagestyle{plain} (by default associated with \chapter{}) will print the page number as the center footer on the first page of each chapter.

5.3.4 Header and Footer with the fancy Style Under the fancyhdr Package*

Similar to the fancyheadings package, there is fancyhdr package that also defines the fancy page style¹. The header and footer commands under the fancyhdr package are slightly different than those under the fancyheadings package. For obtaining the same style (as shown in Table 5.8) under the fancyhdr package, the commands of lines 7–11 in Table 5.8 are to be replaced by the six lines of commands given in Table 5.9.

Table 5.9 Header and footer with the fancy page style under the fancyhdr package

L ^A T _E X input	1	\fancyhead[le,ro]{\textbf{\thepage}}
	2	\fancyhead[re]{\textbf{\leftmark}}
	3	\fancyhead[lo]{\textbf{\rightmark}}
	4	\fancyfoot[le]{\textbf{Engineering Mechanics}}
	5	\fancyfoot[ro]{\textbf{Dilip Datta}}
	6	\fancyfoot[c]{}

¹The header style under the headings page style is predefined, while the header/footer styles under the myheadings and fancy page styles are user-defined.

The `\fancyhead[{}]` and `\fancyfoot[{}]` are, respectively, the commands for headers and footers, whose **l**, **c**, and **r** options stand for left, center, and right aligned header/footer, while **o** and **e** mean odd and even numbered pages, respectively. Accordingly, the combinations **lo** and **le** stand for left alignment on odd and even numbered pages, respectively, while **ro** and **re** mean right alignment on those pages. If no choice is supplied for alignment, headers/footers will be printed in all the three positions (left, center, and right). Similarly, if no choice is supplied for pages, a header/footer will be printed on both odd and even numbered pages. On the other hand, if none of alignment and page is provided, headers/footers will be printed in all the three alignments on both odd and even numbered pages.

5.4 Page Breaking and Adjustment

The application of the `\nopagebreak` command at a point prevents the page breaking at that point. On the other hand, the `\newpage`, `\pagebreak`, `\clearpage`, or `\cleardoublepage` command may be used for printing the remaining contents of a document on a new page by breaking the current page at the point where the command appears. In the case of **twoside** option to `\documentclass[{}]`, `\cleardoublepage` prints the remaining materials of a document from the next odd numbered (right hand) page, even by leaving the previous even numbered (left hand) page blank, if required. In a multi-column page, the `\newpage` and `\pagebreak` commands start a new column instead of a new page. In that case, a new page can be started using `\clearpage` or `\cleardoublepage` only.

The `\raggedbottom` declaration in the preamble puts texts from the top of a page, leaving extra space at the bottom if no material is available to put in that limited space. The `\raggedbottom` declaration is default, except under the **twoside** option to `\documentclass[{}]`. In contrary, the `\flushbottom` declaration makes all pages of the same height adding extra vertical space, if required². Even when `\flushbottom` is in effect, `\newpage` may be used to produce a shortened page (i.e., like `\raggedbottom` page).

5.5 Page Numbering

As seen in Table 1.2 on page 4, by default L^AT_EX assigns a page number in Arabic numeral at the bottom-center of every page, starting with 1 from the very first page of a document. Other numbering styles can be obtained by manual setting, either

²The `\raggedbottom` declaration (default except **twoside** option to `\documentclass[{}]`) puts texts from the top of a page leaving extra space at the bottom if required, while the `\flushbottom` declaration makes all pages of the same height adding extra vertical space if required.

for all pages or for selective pages of a document. A few such styles are explained below:

1. Five types of page numbering are available, which are listed in Table 5.10³.

Table 5.10 Different types of page numbering

Type of numbering	Meaning
arabic	Arabic numerals, like 1, 2, ...
roman	Lowercase Roman numerals, like i, ii, ...
Roman	Uppercase Roman numerals, like I, II, ...
alph	Lowercase English alphabets, like a, b, ...
Alph	Uppercase English alphabets, like A, B, ...

A particular type of numbering can be obtained through `\pagenumbering{anum}`, where `anum` is the required type of numbering as shown in Table 5.10, e.g., `\pagenumbering{alph}` for numbering pages by lowercase English alphabets. The effect would be global if `\pagenumbering{}` is put in the preamble. For local effect, the command can be put inside the `document` environment also, e.g., numbering the pages of the front matter of a book by lowercase Roman numerals, while those of the main matter and back matter by Arabic numerals. If `\pagenumbering{}` is put somewhere inside the `document` environment, the previous numbering style will be altered with effect from the current page onward. Whenever `\pagenumbering{}` is used, the counter of the pages is reset to unity starting from the page containing the command. For effective local implementation, `\pagenumbering{}` should be preceded by one of the `\newpage`, `\clearpage` and `\cleardoublepage` commands, which will terminate the current page and print the remaining materials of a document starting from the next page.

2. By default page numbering starts from unity. Numbering can be started from any other number using `\setcounter{page}{n}` in the preamble, where `n` (an integer) is the desired starting page number of the document.
3. The `\thispagestyle{empty}` command may be used on a page to avoid it from numbering (although the page will not be numbered, it will be counted while numbering the following pages). This may be useful in a report, where number is not to be shown on the first page. If a page is not to be numbered as well as not to be counted, the `\setcounter{page}{n}` command with appropriate value of `n` may be used at the starting of the next page.
4. If none of the pages of a document is to be numbered, the `\pagestyle{empty}` or the `\let\thepage\relax` command may be used in the preamble.

³Five types of numbering of pages and numbered units are **arabic**, **roman**, **Roman**, **alph**, and **Alph**.

Listing and Tabbing Texts

Important matters in a document are usually listed point-wise, either for concise presentation or for making them prominent. Similarly, texts may also need to be tabbed in different columns along the width of a page.

6.1 Listing Texts

There are three listing environments, namely **enumerate**, **itemize**, and **description**. The **enumerate** environment creates a numbered list and the **itemize** environment creates an unnumbered list, while the **description** environment is used to generate a list with user-defined labels. In any of these environments, each individual item is written through an **\item** command, i.e., an item is preceded by an **\item** command. An **\item** command prints an item on a new line/paragraph, and two items are separated by a predefined vertical gap, which can be controlled locally by assigning a suitable value to the **\itemsep** command inside an environment, e.g., **\setlength{\itemsep}{0mm}** for eliminating the vertical gap, while **\setlength{\itemsep}{10mm}** for maintaining a vertical gap of 10 mm.

6.1.1 Numbered Listing Through the **enumerate** Environment

The **enumerate** environment produces a numbered list of items, where the items are numbered by Arabic numerals as shown in Table 6.1. It is also possible to write

Table 6.1 Numbered listing through the **enumerate** environment

L ^A T _E X input	Output
Some states of India are listed below: \begin{enumerate} \item Assam \item Punjab \item Rajasthan. \end{enumerate}	Some states of India are listed below: 1. Assam 2. Punjab 3. Rajasthan.

an `enumerate` environment inside another `enumerate` environment, in which case the inner environment will belong to an `\item` of the outer environment. A maximum of four `enumerate` environments can be nested one inside another for producing a hierarchy of items¹. Such an example is shown in Table 6.2, which also shows how

Table 6.2 Nested numbered listing through the `enumerate` environment

L ^A T _E X input	Output
<pre>Some Asian countries and ... listed below: \begin{enumerate} \item India\label{item:Ind} \begin{enumerate} \item Assam\label{item:Ass} \begin{enumerate} \item Sonitpur\label{item:Sonit} \begin{enumerate} \item Tezpur\label{item:Tez} \item Dhekiajuli \item Balipara \end{enumerate} \item Kamrup \item Cachar \end{enumerate} \item Bihar \item Punjab \end{enumerate} \item Sri Lanka \end{enumerate} Place~\ref{item:Tez} ... district~\ref{item:Sonit} in state~\ref{item:Ass} of country~\ref{item:Ind}.</pre>	Some Asian countries and their various places are listed below: 1. India (a) Assam i. Sonitpur A. Tezpur B. Dhekiajuli C. Balipara ii. Kamrup iii. Cachar (b) Bihar (c) Punjab 2. Sri Lanka Place 1(a)iA belongs to district 1(a)i in state 1a of country 1.

an enumerated item can be labeled and referred through the `\label{}` and `\ref{}` commands, respectively (blank spaces preceding inner lines in the L^AT_EX input are kept only for easy understanding of a loop, otherwise they do not have any sense in L^AT_EX). The default numbering styles in the nested four listings under the `enumerate` environment are 1, 2, ...; (a), (b), ...; i, ii, ... and A, B, ... respectively, while their referring styles are 1, 2, ...; 1a, 1b, ...; 1(a)i, 1(a)ii, ... and 1(a)iA, 1(a)iB, ... respectively.

6.1.1.1 Changing the Numbering Style I*

The default numbering styles in the nested `enumerate` environment can be altered by redefining `\labelenumi`, `\labelenumii`, `\labelenumiii`, and `\labelenumiv`, where `\labelenumi` governs the numbering style in the first `enumerate` environment, `\labelenumii` in the second `enumerate` environment, and so on. Similarly, their default referring styles can be altered by redefining `\theenumi`, `\theenumii`, `\theenumiii`, and `\theenumiv`, respectively. The example of Table 6.2 is reproduced in Table 6.3 on the following page by altering the default numbering and referring styles, where the differences between the two patterns are self-explanatory. The fields `enumi`, `enumii`, `enumiii`, and `enumiv` are, respectively, the counters of the items in four nested

¹A maximum of four `enumerate` environments can be nested for producing a hierarchy of items.

Table 6.3 Altering styles of numbered listing under the **enumerate** environment

L ^A T _E X input	Output
<pre>\documentclass[11pt,a4paper]{article} % \renewcommand{\labelenumi}{\arabic{enumi}.} \renewcommand{\labelenumii}{(\Alph{enumii})} \renewcommand{\labelenumiii}{(\alph{enumiii})} \renewcommand{\labelenumiv}{(\roman{enumiv})} \renewcommand{\theenumi}{\arabic{enumi}} \renewcommand{\theenumii}{(\Alph{enumii})} \renewcommand{\theenumiii}{(\alph{enumiii})} \renewcommand{\theenumiv}{(\roman{enumiv})} % \begin{document} Some Asian countries and ... listed below: \begin{enumerate} \item India\label{item:Ind} \begin{enumerate} \item Assam\label{item:Ass} \begin{enumerate} \item Sonitpur\label{item:Sonit} \begin{enumerate} \item Tezpur\label{item:Tez} \item Dhekiajuli \item Balipara \end{enumerate} \end{enumerate} \item Kamrup \item Cachar \end{enumerate} \item Bihar \item Punjab \end{enumerate} \item Sri Lanka \end{enumerate} Place~\ref{item:Tez} ... district~\ref{item:Sonit} in state~\ref{item:Ass} of country~\ref{item:Ind}. \end{document}</pre>	Some Asian countries and their various places are listed below: 1. India (A) Assam (a) Sonitpur (i) Tezpur (ii) Dhekiajuli (iii) Balipara (b) Kamrup (c) Cachar (B) Bihar (C) Punjab 2. Sri Lanka Town 1(A)(a)(i) belongs to district 1(A)(a) in state 1(A) of country 1.

enumerate environments, while **\arabic{enumi}**., **(\Alph{enumii})**, **(\alph{enumiii})**, and **(\roman{enumiv})** are the numbering styles of those items.

6.1.1.2 Changing the Numbering Style II*

In many cases some fixed texts may need to be included in the numbering of a set of listed items, e.g., Exercise, Question, Note, etc. Such texts can be included in the redefinitions of the **\labelenumi**, **\labelenumii**, **\labelenumiii** and **\labelenumiv** commands. However, even if the numbering style of the items is changed by including some fixed texts, they will still be referred by their serial numbers only, i.e., without the included fixed texts. Table 6.4 on the next page shows an example where the serial numbers of the items in the first **enumerate** environment are preceded by (Q.).

Table 6.4 Numbered listing under the `enumerate` environment mixed with global fixed texts

L^AT_EX input	<pre> \documentclass[11pt,a4paper]{article} \newcommand{\labelenumi}{Q.\theenumi} % \begin{document} The following questions ...\\ Answer Q.\ref{must} and any two from the rest. \begin{enumerate} \item Whether the following statements are true or false?\label{must} \begin{enumerate} \item Water is composed of oxygen and hydrogen. \item Scientific symbol of iron is Hg. \item The value of the gravitational acceleration is 10. \end{enumerate} \item What is photosynthesis? \item What do you mean by magnetism? \item State the Newton's law of motion. \end{enumerate} \end{document} </pre>
Output	The following questions are from General Science: Answer Q.1 and any two from the rest. Q.1. Whether the following statements are true or false? (a) Water is composed of oxygen and hydrogen. (b) Scientific symbol of iron is Hg. (c) The value of the gravitational acceleration is 10. Q.2. What is photosynthesis? Q.3. What do you mean by magnetism? Q.4. State the Newton's law of motion.

If the `\labelenumi`, `\labelenumii`, `\labelenumiii` and `\labelenumiv` commands are redefined in the preamble of a document, their effects will be global. To get different local effects in different `enumerate` environments, the commands should be redefined repeatedly before starting every `enumerate` environment. However, it may not always be convenient to redefine the commands every time, particularly when two or more environments are nested one inside another. Such drawbacks can be overcome through the `enumerate` package, which redefines the `enumerate` environment with an optional argument for specifying its numbering style², e.g., `\begin{enumerate}[Note 1]` for numbering the items of the environment as Note 1, Note 2, etc. The tokens `1`, `i`, `I`, `a` and `A` are reserved for indicating a numbering style. If any of these five tokens appears in the fixed texts of the optional field, it is to be protected by writing it in `{}`. For example, the letter ‘a’ of ‘Lemma’ is to be protected by writing it as `{a}` (i.e., `[Lemm{a} 1]`), otherwise it would be treated as a counter instead of just a fixed letter. Although the numbering styles of the items are changed by including some fixed texts, here also the items are referred by their serial numbers only, i.e., without the included fixed texts. Table 6.5 on the following page shows a document that contains multiple `enumerate` environments with different numbering styles.

²Numbering styles of items of the `enumerate` environment can be changed, including the addition of some fixed texts, either redefining `\labelenumi`, `\labelenumii`, `\labelenumiii` and `\labelenumiv`, or including an optional argument to the environment as defined in the `enumerate` package.

Table 6.5 Numbered listing under the `enumerate` environment mixed with local fixed texts

LaTeX input	Output
<pre>\documentclass[11pt,a4paper]{article} \usepackage{enumerate} % \begin{document} \begin{center}{\bf EXAMPLES}\end{center} \begin{enumerate}[\bf Ex{a}mple 1:]} \item Show that... \item Prove that...\label{item:ex_gr} \item What would be... \end{enumerate} % \begin{center}{\bf PROBLEMS}\end{center} \begin{enumerate}[\bf Problem (a):]} \item Prove that...\label{item:pr_gr} \item Show that... \item What would be... \end{enumerate} % The problem~(\ref{item:pr_gr}) is just an extension of the example~\ref{item:ex_gr}. \end{document}</pre>	EXAMPLES Example 1: Show that... Example 2: Prove that... Example 3: What would be... PROBLEMS Problem (a): Prove that... Problem (b): Show that... Problem (c): What would be... The problem (a) is just an extension of the example 2.

6.1.2 Unnumbered Listing Through the `itemize` Environment

An unnumbered list is produced through the `itemize` environment, an example of which is shown in Table 6.6. Unlike in a numbered list, an item of an unnumbered

Table 6.6 Unnumbered listing through the `itemize` environment

LaTeX input	Output
<pre>\begin{itemize} \item Assam \item Bihar \item Punjab \item Rajasthan. \end{itemize}</pre>	• Assam • Bihar • Punjab • Rajasthan.

list cannot be referred even if it is labeled by a reference key. This is because the item does not have any serial number to refer it. Like the `enumerate` environment, a maximum of four `itemize` environments can be nested one inside another. This is shown in Table 6.7 on the following page, which is the reproduction of the example of Table 6.2 in the `itemize` environments. The default markings of the items in the four nested `itemize` environments are bullet, hyphen, asterisk and dot, respectively.

As in the case of the `enumerate` environment, the default markings in the `itemize` environment can also be altered by redefining the `\labelitemi`, `\labelitemii`, `\labelitemiii` and `\labelitemiv` commands³. Table 6.8 on the next page shows an example where the default markings in two nested `itemize` environments are replaced by the symbols ★ and ▷, respectively. As mentioned in §6.1.1.2 on page 51, if `\labelitemi`, `\labelitemii`,

³Marking styles of items of the `itemize` environment can be changed by redefining the `\labelitemi`, `\labelitemii`, `\labelitemiii` and `\labelitemiv` commands.

Table 6.7 Nested unnumbered listing through the `itemize` environment

L ^A T _E X input	Output
<pre>Some Asian countries and ... are listed below: \begin{itemize} \item India \begin{itemize} \item Assam \begin{itemize} \item Sonitpur \begin{itemize} \item Tezpur \item Dhekiajuli \item Balipara \end{itemize} \item Kamrup \item Cachar \end{itemize} \item Bihar \item Punjab \end{itemize} \item Sri Lanka \end{itemize}</pre>	Some Asian countries and their various places are listed below: • India – Assam * Sonitpur · Tezpur · Dhekiajuli · Balipara * Kamrup * Cachar – Bihar – Punjab • Sri Lanka

Table 6.8 Altering styles of unnumbered listing under the `itemize` environment

L ^A T _E X input	Output
<pre>\renewcommand{\labelitemi}{\$\bigstar\$} \renewcommand{\labelitemii}{\$\rightarrow\$} ... \begin{itemize} \item India \begin{itemize} \item Assam \item Bihar \item Punjab \end{itemize} \item Sri Lanka \end{itemize}</pre>	★ India ▷ Assam ▷ Bihar ▷ Punjab ★ Sri Lanka

etc., are redefined in the preamble of a document, their effects will be global. On the other hand, if they are redefined somewhere inside the `document` environment, their effects will be local only to those `itemize` environments that follow these redefinitions.

6.1.3 Listing with User-Defined Labels Through the `description` Environment

The `description` environment facilitates to prepare a list of items with user-defined labels. Like the `itemize` environment, the `description` environment also does not have any counter, for which its items can not be referred by any serial number. An item in

the **description** environment is labeled through an optional argument to the **\item** command, e.g., **\item[(a)]** will label its item by **(a)** (labeling of items under the **enumerate** or **itemize** environment can also be changed in this way by providing an optional argument to the **\item** command). Table 6.9 shows an application of the **description**

Table 6.9 Listing with user-defined labels through the **description** environment

LaTeX input	Output
<pre>\begin{description} \item[(a)] Assam \item[(b)] Bihar \item[(c)] Punjab \item[(d)] Rajasthan. \end{description}</pre>	(a) Assam (b) Bihar (c) Punjab (d) Rajasthan.

environment, where its items are labeled by **(a)**, **(b)**, etc. The optional argument of **\item[]** can be anything, like (a), (b), (i), (ii), or Rule, Action, etc., which is printed in boldface fonts. The font style of labeling can be changed by redefining the **\descriptionlabel** command⁴, e.g., the **\renewcommand{\descriptionlabel}[1]{\textit{#1}}** command will print the labels in italic fonts (the effect will be global if redefined in the preamble, otherwise local only to the following **description** environments if redefined inside the **document** environment). Like the **enumerate** and **itemize** environments, the **description** environments can also be nested one inside another.

Since items in the **description** environment are labeled by providing label-names in **[]** after the **\item** command, **[]** in the starting of an item, if any, is to be protected by writing in **{}** as **{[]}**, e.g., **\item[Q.1]{[Delhi/Mumbai]}** is the capital of India' for producing 'Q.1 [Delhi/Mumbai] is the capital of India'.

6.1.4 Nesting Different Listing Environments

It is discussed in §6.1.1–6.1.3 that two or more of each of the **enumerate**, **itemize** and **description** environments can be nested one inside another. Nesting of different listing environments is also possible for producing a hierarchy of items. Table 6.10 on the next page shows an **enumerate** environment nested separately with another **enumerate**, an **itemize**, and a **description** environments. By default the items of the main **enumerate** environment are numbered by 1, 2 and 3, respectively. Since the first nested environment is another **enumerate** environment, its items are numbered by (a), (b) and (c), respectively, i.e., by the second level of numbering in the nested **enumerate** environments (refer §6.1.1 for detail). However, since different environments are nested in the second case (an **itemize** environment inside an **enumerate** environment), the items of the **itemize** environment are labeled by its first level of labeling, i.e., by bullet marks. On the other hand, as usual the items of the **description**

⁴The font style of item labeling in the **description** environment can be changed by redefining the **\descriptionlabel** command.

Table 6.10 Nested different listing environments

L ^A T _E X input	Output
<pre>\begin{enumerate} \item SI System \begin{enumerate} \item Metre \item Newton \item Second \end{enumerate} \item MKS System \begin{itemize} \item Metre \item Kilogram \item Second \end{itemize} \item FPS System \begin{description} \item[(i)] Foot \item[(ii)] Pound \item[(iii)] Second \end{description} \end{enumerate}</pre>	1. SI System (a) Metre (b) Newton (c) Second 2. MKS System • Metre • Kilogram • Second 3. FPS System (i) Foot (ii) Pound (iii) Second

environment in the third case are labeled by the supplied texts of (i), (ii) and (iii), respectively. These default patterns of labeling can also be altered as discussed in §6.1.1–6.1.3.

6.1.5 Indentation of Listed Items*

Notice in Tables 6.1, 6.2, 6.3 and 6.7 that the listed items are printed with a predefined indentation on the left side. Sometime space becomes precious seeking to reduce that indentation, which can be done as follows:

- ▷ The size of item indentation in the **enumerate** and **itemize** environments can be adjusted locally by assigning a suitable value to the **leftmargin** option defined in the **enumitem** package. It is to be done at the starting of an environment, e.g., as **\begin{enumerate}[leftmargin=4mm]** or **\begin{itemize}[leftmargin=4mm]** for adjusting the indentation to 4 mm.
- ▷ However, the **enumitem** package conflicts with the **enumerate** package, which redefines the **enumerate** environment to take an optional argument for specifying the numbering style of items as explained in §6.1.1.2 on page 51. Accordingly, if both the provisions (changing numbering style and adjusting indentation) are essential in the same document, an alternative for adjusting item indentation in the **enumerate** environment would be to specify the numbering style (not necessarily to change anything) by incorporating **\hspace{}** with a suitable value, e.g., **\begin{enumerate}[\hspace{0mm}]1.** for numbering in Arabic numerals without any indentation, or **\begin{enumerate}[\hspace{0mm}](a)** for the same job but numbering by lowercase alphabets in a pair of parentheses.

6.2 Tabbing Texts Through the `tabbing` Environment

The `tabbing` environment is used for aligning texts in different columns. The `\=` command is used, usually in the first row, to generate a new column by ending the current column. The `\>` command moves the control to the next column in the subsequent rows. Each row is terminated by a line break command `\` to go to the next row (the last row is not required to be terminated by `\`). Table 6.11 shows a simple two-column

Table 6.11 Tabbing texts in different columns through the `tabbing` environment

LaTeX input	Output
<code>\begin{tabbing}</code>	Potato 12.00
Potato <code>\=</code> 12.00 <code>\</code>	Rice 20.00
Rice <code>\></code> 20.00 <code>\</code>	Oil 60.00
Oil <code>\></code> 60.00 <code>\</code>	Sugar 23.00
Sugar <code>\></code> 23.00	
<code>\end{tabbing}</code>	

example of tabbing through the `tabbing` environment. The two columns are generated in the first row by a `\=` command (one `\=` command separates two columns) and the row is terminated by `\`. The remaining rows are inserted in the same way, but replacing `\=` with `\>`.

6.2.1 Adjusting Column Width in the `tabbing` Environment

The width of a column is fixed based on the length of the entry in the column in that row in which it is generated. If the width is not sufficient to accommodate the entry of that column in any subsequent row, the `\hspace{}` or `\hspace*{}` command can be used in the column generating row to increase the width of the column. This is shown in Table 6.12, where the width of the first column is increased by 0.5 cm using

Table 6.12 Adjusting tabbing column width in the `tabbing` environment through the `\hspace{}` command

LaTeX input	Output
<code>\begin{tabbing}</code>	Breadth (b) = 3.00
Breadth (b) <code>\hspace{0.5cm}</code> <code>\=</code> 3.00 <code>\</code>	Depth (d) = 2.00
Depth (d) <code>\></code> 2.00 <code>\</code>	Height (h) = 4.00
Height (h) <code>\></code> 4.00 <code>\</code>	Volume (V) = bdh = 24.00
Volume (V) <code>\></code> bdh <code>\=</code> 24.00 <code>\</code>	Base Area (A) = bd = 6.00
Base Area (A) <code>\></code> bd <code>\></code> 6.00 <code>\</code>	
<code>\end{tabbing}</code>	

`\hspace{0.5cm}` in the first row. Without the additional space created by `\hspace{}`, the width of the column would not be sufficient to accommodate the entry of that column in the last row. Note that all the columns in a `tabbing` environment are not required to be generated in the first row itself. In Table 6.12, originally two columns are generated in the first row. The necessity of another (the third) column is felt in the fourth row, and hence it is generated there by splitting the second column into two by using a `\=` command. This column could be generated in the first row also,

by adjusting its width through a `\hspace{}` command. In that case, the third column had to be left blank till the fourth row.

Another option for creating columns of required widths and number is to use the `\kill` command. In that case, all the columns are generated in the first row itself, where the entry of a column is the widest entry which appears later in that column. Finally, the row is ended by the `\kill` command, instead of the line breaking command `\`, instructing not to print the row but just to generate the columns. As an example of using the `\kill` command, the list of Table 6.12 is reproduced in Table 6.13.

Table 6.13 Adjusting tabbing column width in the `tabbing` environment using the `\kill` command

LaTeX input	Output
<code>\begin{tabbing}</code>	
Base area (A) <code>\= = 4.00 \= = 24.00\kill</code>	Breadth (b) = 3.00
Breadth (b) <code>\> = 3.00\</code>	Depth (d) = 2.00
Depth (d) <code>\> = 2.00\</code>	Height (h) = 4.00
Height (h) <code>\> = 4.00\</code>	Volume (V) = bdh = 24.00
Volume (V) <code>\> = bdh \> = 24.00\</code>	Base Area (A) = bd = 6.00
Base Area (A) <code>\> = bd \> = 6.00\</code>	
<code>\end{tabbing}</code>	

6.2.2 Adjusting Alignment of Columns in the `tabbing` Environment*

By default the entry of a column in the `tabbing` environment is left aligned. Provision is there for right-aligning the last entry, for which the last entry is to be preceded by the `\` command. Moreover, the `\` command can be used between two pieces of texts of the entry of a column, in which case the first piece of texts is printed right aligned in the previous column. The applications of both the `\` and `\` commands are shown in Table 6.14.

Table 6.14 Aligning tabbing texts in the `tabbing` environment using `\` and `\`

LaTeX input	Output
<code>\begin{tabbing}</code>	
Longest <code>\= Longest \= Longest\kill</code>	a b c
a <code>\> b \> c\</code>	a b c
a <code>\> b \> \c\</code>	a b c
<code>\> a\`b \> c\</code>	a b c
a <code>\> b \> c\</code>	
<code>\end{tabbing}</code>	

Note that `\`, `\` and `\=` cannot be used in the `tabbing` environment for producing accents (refer Table A.1 on page 247 for detail) as they bear different meanings in this environment. Instead of those, `\a``, `\a'` and `\a=` are to be used, e.g., the `\a`{o}`, `\a'{o}`, and `\a={o}` commands in a `tabbing` environment will generate the accents ò, ó, and ô, respectively, which are the same with those usually produced by the `\{o}`, `\'o}` and `\={o}` commands, respectively.